

```
# Import necessary libraries
import pandas as pd
import numpy as np
import json
import re
import time
from datetime import datetime

# Text Preprocessing
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Download necessary NLTK data (run once)
try:
    nltk.data.find('tokenizers/punkt')
except LookupError:
    nltk.download('punkt')
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords')
try:
    nltk.data.find('corpora/wordnet')
except LookupError:
    nltk.download('wordnet')

# Scikit-learn for baseline models and metrics
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.multioutput import MultiOutputClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    roc_auc_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    auc
)
from sklearn.preprocessing import LabelEncoder, MultiLabelBinarizer

# PyTorch and Hugging Face Transformers
import torch
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW # Correct import for AdamW
```

```

from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    get_linear_schedule_with_warmup,
    DistilBertTokenizer, # Example using DistilBERT
    DistilBertForSequenceClassification
)

```

```
# Plotting
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Set random seed for reproducibility
```

```
np.random.seed(42)
```

```
torch.manual_seed(42)
```

```
if torch.cuda.is_available():
```

```
    torch.cuda.manual_seed_all(42)
```

```
print("Libraries imported successfully.")
```

```

➡ [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
Libraries imported successfully.

```

```
# Import necessary libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import json
```

```
import re
```

```
import time
```

```
from datetime import datetime
```

```
# Text Preprocessing
```

```
import nltk
```

```
from nltk.corpus import stopwords
```

```
from nltk.tokenize import word_tokenize
```

```
from nltk.stem import WordNetLemmatizer
```

```
# Download necessary NLTK data (run once)
```

```
try:
```

```
    nltk.data.find('tokenizers/punkt')
```

```
except LookupError:
```

```
    nltk.download('punkt')
```

```
try:
```

```
    nltk.data.find('corpora/stopwords')
```

```
except LookupError:
```

```
nltk.download('stopwords')
try:
    nltk.data.find('corpora/wordnet')
except LookupError:
    nltk.download('wordnet')

# Scikit-learn for baseline models and metrics
# ... (rest of the imports) ...

# Set random seed for reproducibility
# ... (seed setting) ...

print("Libraries imported successfully.")

# --- File Paths ---
# Mount Google Drive
try:
    from google.colab import drive
    drive.mount('/content/drive')
    # Adjust the file paths accordingly
    review_file_path = '/content/drive/My Drive/CMPS 6730 - NLP/FinalProject/yelp_academic_c
    business_file_path = '/content/drive/My Drive/CMPS 6730 - NLP/FinalProject/yelp_academic
    print("Google Drive mounted.")
    print(f"Review data path: {review_file_path}")
    print(f"Business data path: {business_file_path}")
except ModuleNotFoundError:
    # If not in Colab, use the local path directly (adjust if needed)
    review_file_path = r"G:\My Drive\CMPS 6730 - NLP\FinalProject\yelp_academic_dataset_revi
    business_file_path = r"G:\My Drive\CMPS 6730 - NLP\FinalProject\yelp_academic_dataset_bu
    print("Running locally. Ensure the file paths are correct.")
    print(f"Review data path: {review_file_path}")
    print(f"Business data path: {business_file_path}")

# --- Load Review Data in Chunks ---
print(f"\nLoading review data from: {review_file_path}")
start_time = time.time()

all_data = []
chunksize = 100000 # Adjust as needed

for chunk in pd.read_json(review_file_path, lines=True, chunksize=chunksize):
    all_data.append(chunk)

df_reviews = pd.concat(all_data, ignore_index=True)

loading_time = time.time() - start_time
print(f"Review data loaded in {loading_time:.2f} seconds.")
print(f"Review dataset shape: {df_reviews.shape}")
print("\nReview Data Info:")
df_reviews.info()
print("\nReview Data Head:")
```

```
print(df_reviews.head())

# Convert 'date' column to datetime objects
df_reviews['date'] = pd.to_datetime(df_reviews['date'])
print("\n'date' column converted to datetime.")

# --- Load Business Data ---
print(f"\nLoading business data from: {business_file_path}")
start_time = time.time()
try:
    df_business = pd.read_json(business_file_path, lines=True)
except ValueError as e:
    print(f"Error reading business JSON with pandas: {e}. Trying line-by-line.")
    business_data = []
    with open(business_file_path, 'r', encoding='utf-8') as f:
        for line in f:
            try:
                business_data.append(json.loads(line))
            except json.JSONDecodeError:
                print(f"Skipping malformed line in businesses")
                continue
    df_business = pd.DataFrame(business_data)

loading_time = time.time() - start_time
print(f"Business data loaded in {loading_time:.2f} seconds.")
print(f"Business dataset shape: {df_business.shape}")
print("\nBusiness Data Info:")
df_business.info()
print("\nBusiness Data Head:")
print(df_business.head())
print("\nExample Business Categories:", df_business['categories'].iloc[0])
```



0	1616 Chapala St, Ste 2	Santa Barbara	CA	93101
1	87 Grasso Plaza Shopping Center	Affton	MO	63123
2	5255 E Broadway Blvd	Tucson	AZ	85711
3	935 Race St	Philadelphia	PA	19107
4	101 Walnut St	Green Lane	PA	18054

	latitude	longitude	stars	review_count	is_open	\
0	34.426679	-119.711197	5.0	7	0	
1	38.551126	-90.335695	3.0	15	1	
2	32.223236	-110.880452	3.5	22	0	
3	39.955505	-75.155564	4.0	80	1	
4	40.338183	-75.471659	4.5	13	1	

	attributes	\
0	{'ByAppointmentOnly': 'True'}	
1	{'BusinessAcceptsCreditCards': 'True'}	
2	{'BikeParking': 'True', 'BusinessAcceptsCredit...	
3	{'RestaurantsDelivery': 'False', 'OutdoorSeati...	
4	{'BusinessAcceptsCreditCards': 'True', 'Wheelc...	

	categories	\
0	Doctors, Traditional Chinese Medicine, Naturop...	
1	Shipping Centers, Local Services, Notaries, Ma...	
2	Department Stores, Shopping, Fashion, Home & G...	
3	Restaurants, Food, Bubble Tea, Coffee & Tea, B...	
4	Brewpubs, Breweries, Food	

	hours
0	None
1	{'Monday': '0:0-0:0', 'Tuesday': '8:0-18:30', ...
2	{'Monday': '8:0-22:0', 'Tuesday': '8:0-22:0', ...
3	{'Monday': '7:0-20:0', 'Tuesday': '7:0-20:0', ...
4	{'Wednesday': '14:0-22:0', 'Thursday': '16:0-2...

Example Business Categories: Doctors, Traditional Chinese Medicine, Naturopathic/Hol

Filter data based on your criteria (date range AND category)

1. Filter Businesses by Category

```
# Filter by category (must contain 'Restaurants' or 'Food') - ALL locations
df_business['categories'] = df_business['categories'].fillna('')
all_restaurants = df_business[
    df_business['categories'].str.contains('Restaurant|Food', case=False, regex=True)
].copy()
```

```
target_business_ids = set(all_restaurants['business_id'])
print(f"Number of unique target restaurant business IDs: {len(target_business_ids)}")
```

```
if not target_business_ids:
    print("\nWarning: No restaurant business IDs found for the specified location. Check cit
    # Handle this case - perhaps stop execution or proceed with only date filtering?
```

```

# For now, we'll let it proceed, resulting in an empty review dataframe later.

# 2. Filter Reviews by Date
start_date = datetime(2019, 1, 1)
end_date = datetime(2024, 12, 31) # Use end of 2024 for completeness

df_reviews_filtered_date = df_reviews[(df_reviews['date'] >= start_date) & (df_reviews['date'] <= end_date)]
print(f"\nReviews filtered by date ({start_date.date()} to {end_date.date()}). Shape: {df_reviews_filtered_date.shape}")

# 3. Filter Date-Filtered Reviews by Target Business IDs
if target_business_ids:
    df_filtered = df_reviews_filtered_date[
        df_reviews_filtered_date['business_id'].isin(target_business_ids)
    ].copy()
    print(f"Reviews filtered by target business IDs. Shape: {df_filtered.shape}")
else:
    print("\nSkipping business ID filtering as no target IDs were found.")
    df_filtered = pd.DataFrame(columns=df_reviews_filtered_date.columns) # Create empty DataFrame

# Select relevant columns for final processing
# Keep 'business_id' if you might need it later, otherwise drop it
df_processed = df_filtered[['review_id', 'text', 'stars', 'date', 'business_id']].copy()

print(f"\nFinal shape for processing (filtered by date and location/category): {df_processed.shape}")

if df_processed.empty:
    print("\nWARNING: No reviews match the filtering criteria (Date + Location/Category). Stopping execution.")
    # Consider stopping execution here if the dataframe is empty.
else:
    print("\nSelected relevant columns ('review_id', 'text', 'stars', 'date', 'business_id')")
    print(df_processed.head())

# --- Cleanup (Optional: remove intermediate dataframes to save memory) ---
# del df_reviews, df_business, df_reviews_filtered_date, businesses_in_location, restaurants
# import gc
# gc.collect()
# print("\nIntermediate dataframes cleaned up.")

```



Number of unique target restaurant business IDs: 64629

Reviews filtered by date (2019-01-01 to 2024-12-31). Shape: (2111695, 9)

Reviews filtered by target business IDs. Shape: (1525330, 9)

Final shape for processing (filtered by date and location/category): (1525330, 5)

Selected relevant columns ('review_id', 'text', 'stars', 'date', 'business_id').

```

      review_id \
194087  F6VdYuJiefNBfn3HNELv0A
194089  nAMDCKELSKxOhzm9Lpt6Eg
194093  3CmdoGKBZUX3Nb5IfbztMg
203214  -4Nv_JAolCM0gzKM4DZpmQ

```

204271 uObbDRxP_cwJVADckZJDZw

	text	stars	\
194087	The food is INCREDIBLE! We didn't have time to...	5	
194089	We had a great time, and excellent service. Al...	5	
194093	My favorite coffee shop in New Orleans for sur...	5	
203214	Old school circa 1979, the cozy, intimate banq...	4	
204271	Delicious! Tried this place on Saturday and it...	5	

	date	business_id
194087	2019-01-04 02:18:09	z7em5co2qckbAXoDGXynsA
194089	2019-01-06 11:48:21	M0r9lUn2gLFYgIwIfG8-bQ
194093	2019-01-27 15:08:14	itAhmbhH0yQQparfwicjDQ
203214	2019-02-17 20:28:26	-3AooxIkg38UyUdlz5oXdw
204271	2019-02-11 14:59:40	WLiqlfxv_GhFFA5sm878a2w

Start coding or [generate](#) with AI.

```
import pandas as pd
import numpy as np
import nltk
import re # For basic text cleaning
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from collections import defaultdict

# --- Download NLTK data (only needs to be done once) ---
!pip install nltk
import nltk

try:
    nltk.data.find('tokenizers/punkt')
except LookupError: # Changed to catch LookupError
    print("Downloading NLTK 'punkt' tokenizer...")
    nltk.download('punkt', quiet=True)

try:
    nltk.data.find('corpora/stopwords')
except LookupError: # Changed to catch LookupError
    print("Downloading NLTK 'stopwords'...")
    nltk.download('stopwords', quiet=True)

try:
    nltk.data.find('sentiment/vader_lexicon.zip')
except LookupError: # Changed to catch LookupError
    print("Downloading NLTK 'vader_lexicon'...")
    nltk.download('vader_lexicon', quiet=True)

try:
    nltk.data.find('tokenizers/punkt_tab') # Check if the resource is already present
except LookupError:
    print("Downloading NLTK 'punkt_tab'...")
```

```

nltk.download('punkt_tab') # Download specifically 'punkt_tab'

# --- Configuration ---
# Define the path to your processed data (assuming df_processed is loaded elsewhere)
# If df_processed is not loaded, you'll need to load it first, e.g.:
# input_data_file = 'path/to/your/processed_reviews.csv'
# df_processed = pd.read_csv(input_data_file)

# Define where to save the file with sentiment scores
try:
    # Assume Google Drive is mounted if in Colab
    output_sentiment_file = '/content/drive/My Drive/CMPS 6730 - NLP/FinalProject/reviews_wi
    print(f"Output file with sentiment scores will be saved to: {output_sentiment_file}")
except NameError: # If 'drive' object doesn't exist (running locally)
    # Adjust local path if necessary
    output_sentiment_file = r"G:\My Drive\CMPS 6730 - NLP\FinalProject\reviews_with_sentimer
    print(f"Output file with sentiment scores will be saved to: {output_sentiment_file}")

# --- Aspect Keyword Lists (Illustrative Examples - EXPAND THESE SIGNIFICANTLY) ---
# Based on the paper, these lists should be comprehensive.
# Consider synonyms, related terms, common phrases.

# --- Aspect Keyword Lists (EXPANDED - CONTINUE ADDING MORE!) ---
aspect_keywords = {
    'food': [
        # General Food
        'food', 'dish', 'meal', 'plate', 'menu', 'cuisine', 'recipe', 'ingredients', 'eat',
        # Taste & Quality
        'taste', 'flavor', 'delicious', 'tasty', 'yummy', 'savory', 'sweet', 'sour', 'bitter',
        'fresh', 'quality', 'authentic', 'homemade', 'cooked', 'preparation', 'appetizer', '
        'portion', 'serving', 'burnt', 'undercooked', 'overcooked', 'bland', 'seasoning',
        # Specific Items (Examples - Add many more common ones)
        'chicken', 'beef', 'pork', 'fish', 'seafood', 'shrimp', 'crab', 'lobster', 'steak',
        'pizza', 'pasta', 'sushi', 'taco', 'salad', 'soup', 'bread', 'fries', 'rice', 'noodl
        'cake', 'pie', 'ice cream', 'coffee', 'tea', 'drink', 'beverage', 'wine', 'beer', 'c
    ],
    'service': [
        # Staff General
        'service', 'staff', 'server', 'waiter', 'waitress', 'waitstaff', 'host', 'hostess',
        # Staff Behavior
        'friendly', 'attentive', 'helpful', 'professional', 'polite', 'courteous', 'welcomir
        'rude', 'unfriendly', 'inattentive', 'slow', 'ignored', 'forgetful', 'unprofessional
        # Process
        'wait', 'waiting', 'order', 'refill', 'check', 'bill', 'reservation', 'seated', 'pro
        'mistake', 'error', 'issue', 'problem', 'complaint', 'request'
    ],
    'ambiance': [
        # General Feel
        'ambiance', 'atmosphere', 'vibe', 'decor', 'setting', 'environment', 'interior', 'de
        # Sensory

```

```

'music', 'lighting', 'loud', 'noisy', 'quiet', 'sound', 'smell',
# Comfort & Cleanliness
'comfortable', 'cozy', 'relaxing', 'upscale', 'casual', 'romantic', 'view',
'clean', 'dirty', 'tidy', 'messy', 'hygiene', 'restroom', 'bathroom', 'tables',
# Space
'crowded', 'spacious', 'seating', 'booth', 'patio', 'outdoor'
],
'price': [
    # General Cost
    'price', 'cost', 'value', 'money', 'budget', 'bill', 'charge', 'worth', 'pay', 'paid'
    # Affordability
    'cheap', 'expensive', 'affordable', 'overpriced', 'reasonable', 'deal', 'bargain', '
    'value for money', 'rip-off', 'discount', 'coupon', 'special'
],
'context': [
    # Occasion
    'occasion', 'birthday', 'anniversary', 'celebration', 'date', 'romantic', 'special',
    # Company
    'friends', 'family', 'kids', 'children', 'group', 'party', 'business', 'work', 'solo'
    # Time/Meal Type
    'lunch', 'dinner', 'brunch', 'breakfast', 'late night', 'happy hour',
    # Location Related (can overlap with ambiance)
    'location', 'neighborhood', 'parking', 'visit', 'trip', 'tourist'
]
}
# --- Initialize Sentiment Analyzer ---
analyzer = SentimentIntensityAnalyzer()
stop_words = set(stopwords.words('english'))

# --- Helper Functions ---

def clean_text(text):
    """Basic text cleaning."""
    if not isinstance(text, str):
        return ""
    text = text.lower() # Lowercase
    text = re.sub(r'\d+', '', text) # Remove numbers
    text = re.sub(r'[^\w\s]', '', text) # Remove punctuation
    text = text.strip() # Remove leading/trailing whitespace
    return text

def classify_sentence_aspect(sentence, aspect_kw):
    """
    Classifies a sentence into one aspect based on keyword counts.
    Follows the paper's method: assign to aspect with most keywords.
    """
    # Clean and tokenize the sentence
    cleaned_sentence = clean_text(sentence)
    words = word_tokenize(cleaned_sentence)
    # Remove stop words for better keyword matching
    words = [word for word in words if word not in stop_words]

```

```

scores = {aspect: 0 for aspect in aspect_kw}
max_score = 0
best_aspect = 'other' # Default if no keywords match

# Count keywords for each aspect
for aspect, keywords in aspect_kw.items():
    # Use a set for faster checking
    keyword_set = set(keywords)
    # Count occurrences of keywords in the sentence words
    score = sum(1 for word in words if word in keyword_set)
    scores[aspect] = score
    if score > max_score:
        max_score = score
        best_aspect = aspect
    # Handle ties (optional, could assign to 'multiple' or prioritize)
    # elif score == max_score and max_score > 0:
    #     best_aspect = 'multiple' # Or handle differently

# Ensure we only classify if at least one keyword was found
if max_score == 0:
    best_aspect = 'other'

return best_aspect

def get_sentence_sentiment(sentence):
    """
    Calculates the VADER compound sentiment score for a sentence.
    Ranges from -1 (most negative) to +1 (most positive).
    The paper used AFINN (-5 to +5), VADER is a common alternative.
    """
    # VADER's polarity_scores returns dict: {'neg': %, 'neu': %, 'pos': %, 'compound': score}
    vs = analyzer.polarity_scores(sentence)
    return vs['compound'] # Use the compound score as the overall sentiment

def analyze_review_sentiment(review_text, aspect_kw):
    """
    Analyzes a full review text to get weighted sentiment scores per aspect.
    Implements the methodology from the paper.
    """
    if not isinstance(review_text, str) or not review_text.strip():
        # Return default scores for empty or invalid reviews
        return {f'{aspect}_sentiment': 0.0 for aspect in aspect_kw} | {'other_sentiment': 0.0}

    # 1. Tokenize into sentences
    sentences = sent_tokenize(review_text)
    total_sentences = len(sentences)

    if total_sentences == 0:
        # Handle reviews that couldn't be sentence-tokenized
        return {f'{aspect}_sentiment': 0.0 for aspect in aspect_kw} | {'other_sentiment': 0.0}

```

```

# 2. Classify aspect and get sentiment for each sentence
sentence_results = []
for sentence in sentences:
    aspect = classify_sentence_aspect(sentence, aspect_kw)
    sentiment = get_sentence_sentiment(sentence)
    sentence_results.append({'aspect': aspect, 'sentiment': sentiment})

# 3. Aggregate sentiment scores and count sentences per aspect
aspect_sentiments_sum = defaultdict(float)
aspect_sentence_counts = defaultdict(int)

for result in sentence_results:
    aspect = result['aspect']
    sentiment = result['sentiment']
    aspect_sentiments_sum[aspect] += sentiment
    aspect_sentence_counts[aspect] += 1

# 4. Calculate weighted sentiment scores (as per paper's formula)
weighted_scores = {}
all_aspects = list(aspect_kw.keys()) + ['other'] # Include 'other' category

for aspect in all_aspects:
    sum_score = aspect_sentiments_sum[aspect]
    count = aspect_sentence_counts[aspect]

    # Calculate weight (proportion of sentences for this aspect)
    weight = count / total_sentences if total_sentences > 0 else 0

    # Weighted score = Sum * Weight (or proportion)
    # Note: The paper's formula description is slightly ambiguous.
    # Sentiment_ij = Sentiment Score_ij * (# sentences Attribute j / # sentences Attribute i)
    # This implies weighting the *average* score per attribute.
    # Let's calculate both average and weighted sum for clarity.
    # average_score = sum_score / count if count > 0 else 0.0
    # weighted_score_avg_based = average_score * weight # Avg score weighted by proportion

    # Alternative interpretation: Weight the *total* sentiment sum for the aspect
    # This seems more aligned with capturing overall impact.
    weighted_score_sum_based = sum_score * weight

    # Store the weighted score based on the sum interpretation
    weighted_scores[f'{aspect}_{sentiment}'] = weighted_score_sum_based

return weighted_scores

def get_overall_sentiment(review_text): # <-- Add this function here
    """Calculates the overall sentiment score for the entire review."""
    if not isinstance(review_text, str) or not review_text.strip():
        return 0.0 # Return 0 for empty or invalid reviews

```

```

# Use VADER to get the compound sentiment score
scores = analyzer.polarity_scores(review_text)
return scores['compound']

# --- Main Processing Logic ---

# Check if df_processed exists and is populated
# IMPORTANT: Ensure df_processed is loaded before this point if not already in memory.
# Example:
# try:
#     df_processed = pd.read_csv('path/to/your/filtered_reviews.csv')
#     print(f"Loaded {len(df_processed)} reviews.")
# except FileNotFoundError:
#     print("Error: Processed review file not found.")
#     df_processed = pd.DataFrame() # Create empty df to avoid error

if 'df_processed' not in locals() or not isinstance(df_processed, pd.DataFrame) or df_processed.empty:
    print("\nError: The 'df_processed' DataFrame is empty or does not exist.")
    print("Please ensure your data loading step ran successfully.")
    # Optionally, raise an error:
    # raise ValueError("Cannot proceed without data in df_processed.")
else:
    print(f"\nStarting sentiment analysis process on {len(df_processed)} filtered reviews.")

    # Make a copy to avoid modifying the original DataFrame directly
    df_analysis = df_processed.copy()

    # --- Apply Sentiment Analysis ---
    print("Analyzing review sentiments (this may take a while)...")

    # Use .apply() to process the 'text' column of each review
    # The result of apply will be a Series of dictionaries
    sentiment_results = df_analysis['text'].apply(lambda text: analyze_review_sentiment(text))

    # Convert the Series of dictionaries into separate columns in the DataFrame
    # pd.json_normalize is efficient for this
    sentiment_df = pd.json_normalize(sentiment_results)

    # Add these new sentiment score columns to the analysis DataFrame
    df_analysis = pd.concat([df_analysis.reset_index(drop=True), sentiment_df.reset_index(drop=True)], axis=1)

    print("Sentiment analysis complete.")

    # --- Select and Reorder Columns for Output ---
    # Keep original identifiers and add the new sentiment scores
    output_columns = ['review_id', 'business_id', 'date', 'stars', 'text'] + \
        [f'{aspect}_sentiment' for aspect in aspect_keywords] + \
        ['other_sentiment'] # Include 'other' if calculated

    # Ensure all expected columns exist before selecting
    final_columns = [col for col in output_columns if col in df_analysis.columns]
```

```
df_final_output = df_analysis[final_columns]

# --- Export to CSV ---
try:
    # Use index=False to avoid writing the DataFrame index as a column
    df_final_output.to_csv(output_sentiment_file, index=False, encoding='utf-8')
    print(f"\nSuccessfully exported {len(df_final_output)} reviews with sentiment scores")
    print(output_sentiment_file)
    print("\n--- NEXT STEPS ---")
    print("1. Explore the calculated sentiment scores (e.g., correlations with 'stars').")
    print("2. Refine the aspect keyword lists for better classification accuracy.")
    print("3. Consider alternative sentiment lexicons or models if needed.")
    print("4. Use these sentiment scores as features for further analysis or modeling.")

except Exception as e:
    print(f"\nError exporting file: {e}")
    print("Please check the output path and ensure you have write permissions.")

# Optional: Display the first few rows with the new scores
if 'df_final_output' in locals():
    print("\nPreview of DataFrame with Sentiment Scores:")
    print(df_final_output.head())
```

Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nl
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from r
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/dist-package
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nlt
Downloading NLTK 'vader_lexicon'...
Downloading NLTK 'punkt_tab'...
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
Output file with sentiment scores will be saved to: /content/drive/My Drive/CMPS 6730 -

Starting sentiment analysis process on 1525330 filtered reviews.
Analyzing review sentiments (this may take a while)...
Sentiment analysis complete.

Successfully exported 1525330 reviews with sentiment scores to:
/content/drive/My Drive/CMPS 6730 - NLP/FinalProject/reviews_with_sentiment_scores.csv

--- NEXT STEPS ---

1. Explore the calculated sentiment scores (e.g., correlations with 'stars').
2. Refine the aspect keyword lists for better classification accuracy.
3. Consider alternative sentiment lexicons or models if needed.
4. Use these sentiment scores as features for further analysis or modeling.

Preview of DataFrame with Sentiment Scores:

	review_id	business_id	date	stars	\
0	F6VdYuJiefNBfn3HNELv0A	z7em5co2qckbAXoDGXynsA	2019-01-04 02:18:09	5	
1	nAMDCKElSKxOhzm9Lpt6Eg	M0r9lUn2gLFYgIwIfG8-bQ	2019-01-06 11:48:21	5	
2	3CmdoGKBZUX3Nb5IfbztMg	itAhmbhH0yQQparfwicjDQ	2019-01-27 15:08:14	5	
3	-4Nv_JAolCM0gzKM4DZpmQ	-3AooxIkq38UyUdlz5oXdw	2019-02-17 20:28:26	4	
4	uObbDRxP_cwJVADckZJDZw	WLiqfxv_GhFFA5sm878a2w	2019-02-11 14:59:40	5	

	text	food_sentiment	\
0	The food is INCREDIBLE! We didn't have time to...	2.15690	
1	We had a great time, and excellent service. Al...	0.07955	
2	My favorite coffee shop in New Orleans for sur...	1.60730	
3	Old school circa 1979, the cozy, intimate banq...	0.00000	
4	Delicious! Tried this place on Saturday and it...	0.30570	

	service_sentiment	ambiance_sentiment	price_sentiment	context_sentiment	\
0	0.026411	0.0	0.0	0.0000	
1	0.803550	0.0	0.0	0.0000	
2	0.000000	0.0	0.0	0.0000	
3	0.000000	0.0	0.0	0.2202	
4	0.000000	0.0	0.0	0.0000	

	other_sentiment
0	0.423467
1	0.123175
2	0.000000
3	0.000000
4	0.447250

```
# Initialize lemmatizer and stopwords
```

```
import nltk
```

```
from nltk.corpus import stopwords
```

```
from nltk.tokenize import word_tokenize
```

```
from nltk.stem import WordNetLemmatizer
```

```
import re
```

```
import time
```

```
# Download the necessary NLTK data if it's not already present
```

```
try:
```

```
    nltk.data.find('tokenizers/punkt_tab')
```

```
except LookupError:
```

```
    nltk.download('punkt_tab')
```

```
lemmatizer = WordNetLemmatizer()
```

```
stop_words = set(stopwords.words('english'))
```

```
def preprocess_text(text):
```

```
    """Cleans and preprocesses text data."""
```

```
    if not isinstance(text, str):
```

```
        return ""
```

```
    # Lowercase
```

```
    text = text.lower()
```

```
    # Remove URLs
```

```
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
```

```
    # Remove user @ references and '#' from tweet
```

```
    text = re.sub(r'\@w+|\#','', text)
```

```
    # Remove punctuation
```

```
    text = re.sub(r'[^\w\s]','', text)
```

```

# Tokenize
tokens = word_tokenize(text)
# Remove stopwords and lemmatize
processed_tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in stop_wc]
return ' '.join(processed_tokens)

# Apply preprocessing (can take time on large datasets)
print("\nStarting text preprocessing...")
start_time = time.time()
df_processed['processed_text'] = df_processed['text'].apply(preprocess_text)
processing_time = time.time() - start_time
print(f"Text preprocessing completed in {processing_time:.2f} seconds.")

# Display some processed text examples
print("\nOriginal vs Processed Text Examples:")
for i in range(3):
    print(f"--- Example {i+1} ---")
    print("Original:", df_processed['text'].iloc[i][:200] + "...") # Show first 200 chars
    print("Processed:", df_processed['processed_text'].iloc[i])
    print("-" * 20)

# Drop rows where processed text is empty
df_processed = df_processed[df_processed['processed_text'] != '']
print(f"\nShape after removing empty processed texts: {df_processed.shape}")

```



Starting text preprocessing...

Text preprocessing completed in 710.21 seconds.

Original vs Processed Text Examples:

--- Example 1 ---

Original: The food is INCREDIBLE! We didn't have time to get any of the cocktails, but t
Processed: food incredible didnt time get cocktail pretty creative classy one menu going

--- Example 2 ---

Original: We had a great time, and excellent service. All food items were fresh, unique,
Except for the ice cream which was exceptionally good. The waitress wa...
Processed: great time excellent service food item fresh unique high quality nothing appe

--- Example 3 ---

Original: My favorite coffee shop in New Orleans for sure! Perfect to study (quiet envir
Processed: favorite coffee shop new orleans sure perfect study quiet environment super r

Shape after removing empty processed texts: (1525313, 6)



```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from nltk.sentiment.vader import SentimentIntensityAnalyzer # Import SentimentIntensityAnal

```

```

# Initialize SentimentIntensityAnalyzer (if not already initialized)
analyzer = SentimentIntensityAnalyzer()

def get_overall_sentiment(review_text): # <-- Add this function here
    """Calculates the overall sentiment score for the entire review."""
    if not isinstance(review_text, str) or not review_text.strip():
        return 0.0 # Return 0 for empty or invalid reviews

    # Use VADER to get the compound sentiment score
    scores = analyzer.polarity_scores(review_text)
    return scores['compound']

# Assuming df_processed is already loaded and contains the preprocessed text data in the 'pr

# 1. Load or calculate sentiment scores if not in df_processed
try:
    # If sentiment scores are already in df_processed, use them
    sentiment_columns = ['food_sentiment', 'service_sentiment', 'ambiance_sentiment', 'price
    Y = df_processed[sentiment_columns].copy() # Use .copy() to avoid SettingWithCopyWarnin
except KeyError:
    print("Sentiment score columns not found in df_processed. Calculating or loading them...
    # Calculate overall sentiment for each review
    df_processed['overall_sentiment'] = df_processed['text'].apply(get_overall_sentiment)
    # Option 2: Load from a separate file (if you saved them earlier)
    from google.colab import drive
    drive.mount('/content/drive') # Mount Google Drive
    sentiment_file_path = '/content/drive/My Drive/CMPS 6730 - NLP/FinalProject/reviews_with

    sentiment_df = pd.read_csv(sentiment_file_path)
    # Merge sentiment scores with the original dataframe based on a common column (e.g., 're
    df_processed = pd.merge(
        df_processed,
        sentiment_df[['review_id', 'food_sentiment', 'service_sentiment', 'ambiance_sentimer
        on='review_id',
        how='left'
    )
    Y = df_processed[['food_sentiment', 'service_sentiment', 'ambiance_sentiment', 'price_se

# --- 2. Correct sentiment binning using fixed thresholds ---

# Define fixed VADER thresholds and corresponding integer labels
# Bins: (-inf, -0.05], (-0.05, 0.05), [0.05, +inf)
# We add +/- infinity to ensure all values are covered.
bin_edges = [-float('inf'), -0.05, 0.05, float('inf')]
# Assign integer labels: 0 for Negative, 1 for Neutral, 2 for Positive
bin_labels = [0, 1, 2]
label_names = ['negative', 'neutral', 'positive'] # For potential use in reports if needed

print(f"Using fixed bin edges: {bin_edges}")
print(f"Assigning integer labels: {bin_labels} ({', '.join(label_names)})")

```

```

# Ensure Y is a DataFrame (it should be from previous steps)
if not isinstance(Y, pd.DataFrame):
    raise TypeError("Y should be a pandas DataFrame at this stage.")

# Create a new DataFrame for binned labels to avoid modifying Y inplace initially
Y_binned = pd.DataFrame(index=Y.index)

for column in Y.columns:
    # Cast column values to float64 explicitly to avoid dtype issues
    # Use .loc to avoid SettingWithCopyWarning if Y is a slice
    Y_column_float = Y.loc[:, column].astype(float)

    # Use pd.cut for binning with fixed edges
    # include_lowest=True: includes the lowest value (-inf) in the first bin
    # right=True (default): bins are (edge1, edge2], except first which is [edge1, edge2] du
    # If you want bins like [edge1, edge2), use right=False
    binned_data = pd.cut(
        Y_column_float,
        bins=bin_edges,
        labels=bin_labels,
        include_lowest=True,
        right=True # Standard VADER thresholds often use >= 0.05 for positive, <= -0.05 for
    )

    # Check for NaNs introduced by binning (shouldn't happen with inf edges, but good practi
    if binned_data.isnull().any():
        print(f"Warning: NaNs found in '{column}' after binning. Check original data.")
        # Handle NaNs if necessary, e.g., fill with neutral=1 or drop rows
        # binned_data = binned_data.fillna(1) # Example: Fill NaN with Neutral

    # Assign the binned data (as integers) to the new DataFrame
    Y_binned[column] = binned_data.astype(int)

# Replace the original Y with the binned version
Y = Y_binned
print("\nSentiment scores binned using fixed thresholds:")
print(Y.head())
print("\nValue counts for 'food_sentiment' (example):")
print(Y['food_sentiment'].value_counts())

# --- End of binning modification ---

# 3. Prepare data for training and testing
# Assuming 'processed_text' column contains preprocessed text data
X = df_processed['processed_text'].copy() # Use .copy() to ensure a deep copy

# Get the first sentiment column for stratification
stratify_column = Y.iloc[:, 0] # Assuming Y is a DataFrame

# Split data into training and testing sets
X_train, X_test, Y_train, Y_test = train_test_split(

```

```

X, Y, test_size=0.2, random_state=42, stratify=stratify_column
)

# Print dataset sizes
print(f"Training set size: {len(X_train)}")
print(f"Testing set size: {len(X_test)}")

# ... (rest of the code, including TF-IDF, tokenization, and model training) ...

➡ Sentiment score columns not found in df_processed. Calculating or loading them...
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mour
Using fixed bin edges: [-inf, -0.05, 0.05, inf]
Assigning integer labels: [0, 1, 2] (negative, neutral, positive)

Sentiment scores binned using fixed thresholds:

```

	food_sentiment	service_sentiment	ambiance_sentiment	price_sentiment	\
0	2	1	1	1	
1	2	2	1	1	
2	2	1	1	1	
3	1	1	1	1	
4	2	1	1	1	

	context_sentiment	overall_sentiment
0	1	2
1	1	2
2	1	2
3	2	2
4	1	2


```

Value counts for 'food_sentiment' (example):
food_sentiment
2    1019023
1     385113
0     121177
Name: count, dtype: int64
Training set size: 1220250
Testing set size: 305063

```

```

# --- Import necessary libraries ---
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer # Import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.multioutput import MultiOutputClassifier
from sklearn.metrics import classification_report, precision_recall_fscore_support, accuracy

# Assuming df_processed is already loaded and contains the preprocessed text data in the 'pr

# 1. Load or calculate sentiment scores if not in df_processed
try:

```

```

# If sentiment scores are already in df_processed, use them
sentiment_columns = ['food_sentiment', 'service_sentiment', 'ambiance_sentiment', 'price_s
Y = df_processed[sentiment_columns].copy() # Use .copy() to avoid SettingWithCopyWarnin
except KeyError:
    print("Sentiment score columns not found in df_processed. Calculating or loading them...
    # Option 2: Load from a separate file (if you saved them earlier)
    from google.colab import drive
    drive.mount('/content/drive') # Mount Google Drive
    sentiment_file_path = '/content/drive/My Drive/CMPS 6730 - NLP/FinalProject/reviews_with

sentiment_df = pd.read_csv(sentiment_file_path)
# Merge sentiment scores with the original dataframe based on a common column (e.g., 're
df_processed = pd.merge(
    df_processed,
    sentiment_df[['review_id', 'food_sentiment', 'service_sentiment', 'ambiance_sentimen
    on='review_id',
    how='left'
)
Y = df_processed[['food_sentiment', 'service_sentiment', 'ambiance_sentiment', 'price_se

# --- 2. Correct sentiment binning using fixed thresholds ---

# Define fixed VADER thresholds and corresponding integer labels
# Bins: (-inf, -0.05], (-0.05, 0.05), [0.05, +inf)
# We add +/- infinity to ensure all values are covered.
bin_edges = [-float('inf'), -0.05, 0.05, float('inf')]
# Assign integer labels: 0 for Negative, 1 for Neutral, 2 for Positive
bin_labels = [0, 1, 2]
label_names = ['negative', 'neutral', 'positive'] # For potential use in reports if needed

print(f"Using fixed bin edges: {bin_edges}")
print(f"Assigning integer labels: {bin_labels} ({', '.join(label_names)})")

# Ensure Y is a DataFrame (it should be from previous steps)
if not isinstance(Y, pd.DataFrame):
    raise TypeError("Y should be a pandas DataFrame at this stage.")

# Create a new DataFrame for binned labels to avoid modifying Y inplace initially
Y_binned = pd.DataFrame(index=Y.index)

for column in Y.columns:
    # Cast column values to float64 explicitly to avoid dtype issues
    # Use .loc to avoid SettingWithCopyWarning if Y is a slice
    Y_column_float = Y.loc[:, column].astype(float)

    # Use pd.cut for binning with fixed edges
    # include_lowest=True: includes the lowest value (-inf) in the first bin
    # right=True (default): bins are (edge1, edge2], except first which is [edge1, edge2] du
    # If you want bins like [edge1, edge2), use right=False
    binned_data = pd.cut(
        Y_column_float,

```

```

    bins=bin_edges,
    labels=bin_labels,
    include_lowest=True,
    right=True # Standard VADER thresholds often use >= 0.05 for positive, <= -0.05 for
)

# Check for NaNs introduced by binning (shouldn't happen with inf edges, but good practice)
if binned_data.isnull().any():
    print(f"Warning: NaNs found in '{column}' after binning. Check original data.")
    # Handle NaNs if necessary, e.g., fill with neutral=1 or drop rows
    # binned_data = binned_data.fillna(1) # Example: Fill NaN with Neutral

# Assign the binned data (as integers) to the new DataFrame
Y_binned[column] = binned_data.astype(int)

# Replace the original Y with the binned version
Y = Y_binned
print("\nSentiment scores binned using fixed thresholds:")
print(Y.head())
print("\nValue counts for 'food_sentiment' (example):")
print(Y['food_sentiment'].value_counts())

# --- End of binning modification ---

# 3. Prepare data for training and testing
# Assuming 'processed_text' column contains preprocessed text data
X = df_processed['processed_text'].copy() # Use .copy() to ensure a deep copy

# Get the first sentiment column for stratification
stratify_column = Y.iloc[:, 0] # Assuming Y is a DataFrame

# Split data into training and testing sets
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42, stratify=stratify_column
)

# Print dataset sizes
print(f"Training set size: {len(X_train)}")
print(f"Testing set size: {len(X_test)}")

# --- TF-IDF Vectorization ---
# Create a TF-IDF vectorizer
vectorizer = TfidfVectorizer(max_features=5000) # Adjust max_features as needed

# Fit the vectorizer to the training data and transform it
X_train_tfidf = vectorizer.fit_transform(X_train)

# Transform the testing data using the fitted vectorizer
X_test_tfidf = vectorizer.transform(X_test)

```

```

# --- Baseline Model Training and Evaluation ---
# Using OneVsRestClassifier: Trains one classifier per class (or per output)
# Here, we wrap Logistic Regression. One LR model will be trained for each aspect.
print("\nTraining Baseline Model (Logistic Regression with OneVsRest)...")

# Create a base estimator
log_reg = LogisticRegression(solver='liblinear', random_state=42) # Liblinear often good for

# Wrap it with OneVsRestClassifier for multi-label/multi-output scenario
# It trains one classifier per column in Y
baseline_model = MultiOutputClassifier(log_reg) # Simpler API for multi-output integer target

# Train the model
baseline_model.fit(X_train_tfidf, Y_train) # Use the numeric labels DataFrame/Array

# Make predictions on the test set
Y_pred_baseline = baseline_model.predict(X_test_tfidf)

# --- Baseline Evaluation ---
# Assuming 'aspects' and 'sentiment_classes' are defined (from your Transformer code)

baseline_report = {} # To store results in a dictionary
print("\n--- Baseline (Logistic Regression) Classification Report (Per Aspect) ---")

baseline_f1_scores = {} # Store F1-scores per aspect
all_true_flat_baseline = [] # To store flattened true labels for overall metrics
all_pred_flat_baseline = [] # To store flattened predicted labels for overall metrics

# Add these lines before the loop in your code:
aspects = ['food', 'service', 'ambiance', 'price', 'context', 'overall_sentiment']
sentiment_classes = ['negative', 'neutral', 'positive']

for i, aspect_name in enumerate(aspects):
    print(f"\n--- Aspect: {aspect_name} ---")
    true_labels = Y_test.iloc[:, i].values # Get true labels for this aspect
    pred_labels = Y_pred_baseline[:, i] # Get predicted labels for this aspect

    # Get unique labels in pred_labels and true_labels
    unique_labels = np.unique(np.concatenate((pred_labels, true_labels)))

    # Filter target_names to include only the present labels
    present_target_names = [name for idx, name in enumerate(sentiment_classes) if idx in uni

report = classification_report(true_labels, pred_labels, target_names=present_target_names)
print(report)
precision, recall, f1, _ = precision_recall_fscore_support(true_labels, pred_labels, average='micro')
accuracy = accuracy_score(true_labels, pred_labels)
baseline_report[aspect_name] = {'precision': precision, 'recall': recall, 'f1-score': f1}
baseline_f1_scores[aspect_name] = f1 # Store F1-score for this aspect

```

```
all_true_flat_baseline.extend(true_labels) # Extend for overall metrics
all_pred_flat_baseline.extend(pred_labels) # Extend for overall metrics
```

```
# Overall Micro and Macro Averages for Baseline
```

```
print("\n--- Baseline Overall Micro/Macro Averages ---")
```

```
precision_micro_b, recall_micro_b, f1_micro_b, _ = precision_recall_fscore_support(all_true_
precision_macro_b, recall_macro_b, f1_macro_b, _ = precision_recall_fscore_support(all_true_
print(f"Micro Average: Precision={precision_micro_b:.4f}, Recall={recall_micro_b:.4f}, F1-Score={f1_micro_b:.4f}")
print(f"Macro Average: Precision={precision_macro_b:.4f}, Recall={recall_macro_b:.4f}, F1-Score={f1_macro_b:.4f}")
```

```
baseline_report['overall_micro'] = {'precision': precision_micro_b, 'recall': recall_micro_b, 'f1-score': f1_micro_b}
baseline_report['overall_macro'] = {'precision': precision_macro_b, 'recall': recall_macro_b, 'f1-score': f1_macro_b}
```

➡ Using fixed bin edges: [-inf, -0.05, 0.05, inf]
Assigning integer labels: [0, 1, 2] (negative, neutral, positive)

Sentiment scores binned using fixed thresholds:

	food_sentiment	service_sentiment	ambiance_sentiment	price_sentiment	\
0	2	1	1	1	
1	2	2	1	1	
2	2	1	1	1	
3	1	1	1	1	
4	2	1	1	1	

	context_sentiment	overall_sentiment
0	1	2
1	1	2
2	1	2
3	2	2
4	1	2

Value counts for 'food_sentiment' (example):

```
food_sentiment
2    1019023
1     385113
0     121177
```

Name: count, dtype: int64

Training set size: 1220250

Testing set size: 305063

Training Baseline Model (Logistic Regression with OneVsRest)...

--- Baseline (Logistic Regression) Classification Report (Per Aspect) ---

--- Aspect: food ---

	precision	recall	f1-score	support
negative	0.64	0.29	0.40	24235
neutral	0.72	0.68	0.70	77023
positive	0.85	0.92	0.89	203805
accuracy			0.81	305063
macro avg	0.74	0.63	0.66	305063
weighted avg	0.80	0.81	0.80	305063

```

--- Aspect: service ---
      precision    recall  f1-score   support

negative    0.71     0.52     0.60     25208
neutral     0.81     0.90     0.85    187376
positive    0.76     0.64     0.70     92479

accuracy                   0.79    305063
macro avg    0.76     0.69     0.72    305063
weighted avg    0.79     0.79     0.79    305063

```

```

--- Aspect: ambiance ---
      precision    recall  f1-score   support

```

```

# --- Import necessary libraries ---
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer # Import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.multioutput import MultiOutputClassifier
from sklearn.metrics import classification_report, precision_recall_fscore_support, accuracy
import torch
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW # Correct import for AdamW
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    get_linear_schedule_with_warmup,
    DistilBertTokenizer, # Example using DistilBERT
    DistilBertForSequenceClassification
)
import time
from nltk.sentiment.vader import SentimentIntensityAnalyzer

# Initialize SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()

# Assuming df_processed is already loaded and contains the preprocessed text data in the 'pr

# 1. Load or calculate sentiment scores if not in df_processed
try:
    # If sentiment scores are already in df_processed, use them
    sentiment_columns = ['food_sentiment', 'service_sentiment', 'ambiance_sentiment', 'price
    Y = df_processed[sentiment_columns].copy() # Use .copy() to avoid SettingWithCopyWarnin
except KeyError:
    print("Sentiment score columns not found in df_processed. Calculating or loading them...
    # Option 2: Load from a separate file (if you saved them earlier)
    from google.colab import drive
    drive.mount('/content/drive') # Mount Google Drive

```

```

sentiment_file_path = '/content/drive/My Drive/CMPS 6730 - NLP/FinalProject/reviews_with

sentiment_df = pd.read_csv(sentiment_file_path)
# Merge sentiment scores with the original dataframe based on a common column (e.g., 'review_id')
df_processed = pd.merge(
    df_processed,
    sentiment_df[['review_id', 'food_sentiment', 'service_sentiment', 'ambiance_sentiment']],
    on='review_id',
    how='left'
)
Y = df_processed[['food_sentiment', 'service_sentiment', 'ambiance_sentiment', 'price_sentiment']]

# Calculate overall sentiment for each review
df_processed['overall_sentiment'] = df_processed['text'].apply(get_overall_sentiment)

# Now include 'overall_sentiment' in your target columns
Y = df_processed[['food_sentiment', 'service_sentiment', 'ambiance_sentiment', 'price_sentiment', 'overall_sentiment']]

# --- 2. Correct sentiment binning using fixed thresholds ---

# Define fixed VADER thresholds and corresponding integer labels
# Bins: (-inf, -0.05], (-0.05, 0.05), [0.05, +inf)
# We add +/- infinity to ensure all values are covered.
bin_edges = [-float('inf'), -0.05, 0.05, float('inf')]
# Assign integer labels: 0 for Negative, 1 for Neutral, 2 for Positive
bin_labels = [0, 1, 2]
label_names = ['negative', 'neutral', 'positive'] # For potential use in reports if needed

print(f"Using fixed bin edges: {bin_edges}")
print(f"Assigning integer labels: {bin_labels} ({', '.join(label_names)})")

# Ensure Y is a DataFrame (it should be from previous steps)
if not isinstance(Y, pd.DataFrame):
    raise TypeError("Y should be a pandas DataFrame at this stage.")

# Create a new DataFrame for binned labels to avoid modifying Y inplace initially
Y_binned = pd.DataFrame(index=Y.index)

for column in Y.columns:
    # Cast column values to float64 explicitly to avoid dtype issues
    # Use .loc to avoid SettingWithCopyWarning if Y is a slice
    Y_column_float = Y.loc[:, column].astype(float)

    # Use pd.cut for binning with fixed edges
    # include_lowest=True: includes the lowest value (-inf) in the first bin
    # right=True (default): bins are (edge1, edge2], except first which is [edge1, edge2] due to -inf
    # If you want bins like [edge1, edge2), use right=False
    binned_data = pd.cut(
        Y_column_float,
        bins=bin_edges,
        labels=bin_labels,

```

```

        include_lowest=True,
        right=True # Standard VADER thresholds often use >= 0.05 for positive, <= -0.05 for
    )

    # Check for NaNs introduced by binning (shouldn't happen with inf edges, but good practice)
    if binned_data.isnull().any():
        print(f"Warning: NaNs found in '{column}' after binning. Check original data.")
        # Handle NaNs if necessary, e.g., fill with neutral=1 or drop rows
        # binned_data = binned_data.fillna(1) # Example: Fill NaN with Neutral

    # Assign the binned data (as integers) to the new DataFrame
    Y_binned[column] = binned_data.astype(int)

# Replace the original Y with the binned version
Y = Y_binned
print("\nSentiment scores binned using fixed thresholds:")
print(Y.head())
print("\nValue counts for 'food_sentiment' (example):")
print(Y['food_sentiment'].value_counts())

# --- End of binning modification ---

# 3. Prepare data for training and testing
# Assuming 'processed_text' column contains preprocessed text data
X = df_processed['processed_text'].copy() # Use .copy() to ensure a deep copy

# Get the first sentiment column for stratification
stratify_column = Y.iloc[:, 0] # Assuming Y is a DataFrame

# Split data into training and testing sets
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42, stratify=stratify_column
)

# Print dataset sizes
print(f"Training set size: {len(X_train)}")
print(f"Testing set size: {len(X_test)}")

# --- Tokenization ---
# Initialize tokenizer (if not done earlier)
model_name = 'distilbert-base-uncased' # Or your preferred model name
tokenizer = DistilBertTokenizer.from_pretrained(model_name)

# Tokenize and encode the text data
train_encodings = tokenizer(X_train.tolist(), truncation=True, padding=True)
test_encodings = tokenizer(X_test.tolist(), truncation=True, padding=True)

# --- Convert Y_train and Y_test to NumPy arrays ---
Y_train_np = Y_train.values # Assuming Y_train is a pandas DataFrame
Y_test_np = Y_test.values # Assuming Y_test is a pandas DataFrame

```

```

# --- Define PyTorch Dataset ---
class YelpAspectDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        # Ensure labels are shaped correctly (N_samples, N_aspects)
        self.labels = labels

    def __getitem__(self, idx):
        # item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        # Handle cases where encodings might be lists of tensors or similar structures
        item = {}
        for key, val in self.encodings.items():
            # Ensure the value associated with the key is indexable and convert to tensor
            if isinstance(val, list):
                item[key] = torch.tensor(val[idx])
            else: # Assuming it might already be a tensor or numpy array
                item[key] = torch.tensor(val[idx]).clone().detach() # Make sure it's a tensor

        # Labels should be FloatTensor for BCEWithLogitsLoss or LongTensor for CrossEntropyLoss
        # For multi-label (predicting probability for each class per aspect) or multi-output
        # Let's assume multi-output: predicting one class (0, 1, 2) per aspect. Labels are 1D
        # Shape should be (num_aspects,) for a single item
        item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

    def __len__(self):
        # Use the number of samples in one of the encoding keys (e.g., 'input_ids')
        return len(self.encodings['input_ids'])

# Create datasets
train_dataset = YelpAspectDataset(train_encodings, Y_train_np)
test_dataset = YelpAspectDataset(test_encodings, Y_test_np)

# --- Define Model ---
# We need a multi-output classification head.
# Load pre-trained DistilBERT and modify the classifier layer.
aspects = ['food', 'service', 'ambiance', 'price', 'context', 'overall'] # Define your aspects
sentiment_classes = ['negative', 'neutral', 'positive'] # Define sentiment classes

num_aspects = len(aspects)
num_classes_per_aspect = len(sentiment_classes) # 3 classes: positive, negative, neutral

# Load the base model
model = DistilBertForSequenceClassification.from_pretrained(
    model_name,
    num_labels=num_aspects * num_classes_per_aspect # TEMPORARY - see below
)

```

```

# *** IMPORTANT: Modifying the Classifier Head for Multi-Output ***
# The standard `DistilBertForSequenceClassification` has ONE output layer for num_labels.
# For multi-output (one prediction per aspect), we need to customize.
# Option 1: Treat as N independent classification tasks (simpler to implement with standard
# Option 2: Create N classification heads (more complex custom model)
# Option 3: Reshape the output of a single large head (requires careful loss calculation)

# Let's try Option 3 conceptually: A single head predicting scores for all aspect-class comb
# Then reshape and calculate loss per aspect.
# The output dimension should be num_aspects * num_classes_per_aspect
# The final layer needs to be replaced or adapted.

# Get the original classifier's input dimension
original_classifier_in_features = model.classifier.in_features

# Replace the classifier head. Output size is num_aspects * num_classes (e.g., 5 * 3 = 15)
model.classifier = torch.nn.Linear(original_classifier_in_features, num_aspects * num_classes)
# The pre_classifier layer might also need adjustment if present (DistilBERT has one)
if hasattr(model, 'pre_classifier') and model.pre_classifier is not None:
    original_pre_classifier_in_features = model.pre_classifier.in_features
    model.pre_classifier = torch.nn.Linear(original_pre_classifier_in_features, original_pre_classifier_out_features)
else:
    print("Model does not have a separate pre_classifier layer.")

print(f"\nCustomized DistilBERT model loaded. Output layer size: {model.classifier.out_features}")

# --- Training Setup ---
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
print(f"Using device: {device}")
model.to(device)

# Dataloaders
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True) # Adjust batch_size to fit memory
test_loader = DataLoader(test_dataset, batch_size=32) # Larger batch size for evaluation is better

# Optimizer and Scheduler
optimizer = AdamW(model.parameters(), lr=5e-5) # Typical learning rate for fine-tuning
num_epochs = 3 # Adjust as needed (start small)
num_training_steps = num_epochs * len(train_loader)
lr_scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=num_training_steps)

# Loss Function: CrossEntropyLoss is suitable for multi-class classification (per aspect)
loss_fn = torch.nn.CrossEntropyLoss()

# --- Training Loop ---
print("\nStarting Transformer Model Training...")
model.train()
for epoch in range(num_epochs):
    print(f"--- Epoch {epoch + 1}/{num_epochs} ---")
    epoch_start_time = time.time()
    total_loss = 0

```

```

for i, batch in enumerate(train_loader):
    optimizer.zero_grad()
    # Move batch to device
    input_ids = batch['input_ids'].to(device)
    attention_mask = batch['attention_mask'].to(device)
    labels = batch['labels'].to(device) # Shape: (batch_size, num_aspects)

    # Forward pass
    outputs = model(input_ids=input_ids, attention_mask=attention_mask)
    logits = outputs.logits # Shape: (batch_size, num_aspects * num_classes_per_aspect)

    # Calculate loss: Reshape logits and labels for CrossEntropyLoss
    # Logits need to be (batch_size * num_aspects, num_classes_per_aspect)
    # Labels need to be (batch_size * num_aspects,)
    reshaped_logits = logits.view(-1, num_classes_per_aspect) # (batch * aspects, class
    reshaped_labels = labels.view(-1) # (batch * aspects,)

    loss = loss_fn(reshaped_logits, reshaped_labels)
    total_loss += loss.item()

    loss.backward()
    optimizer.step()
    lr_scheduler.step()

    if (i + 1) % 100 == 0: # Print progress every 100 batches
        print(f" Batch {i + 1}/{len(train_loader)}, Loss: {loss.item():.4f}")

avg_train_loss = total_loss / len(train_loader)
epoch_time = time.time() - epoch_start_time
print(f"Epoch {epoch + 1} completed in {epoch_time:.2f}s. Average Training Loss: {avg_tr

print("Transformer training finished.")
# Consider saving the trained model
# model.save_pretrained('./my_aspect_sentiment_model')
# tokenizer.save_pretrained('./my_aspect_sentiment_model')

```



Using fixed bin edges: [-inf, -0.05, 0.05, inf]

Assigning integer labels: [0, 1, 2] (negative, neutral, positive)

Sentiment scores binned using fixed thresholds:

	food_sentiment	service_sentiment	ambiance_sentiment	price_sentiment	\
0	2	1	1	1	
1	2	2	1	1	
2	2	1	1	1	
3	1	1	1	1	
4	2	1	1	1	

	context_sentiment	overall_sentiment
0	1	2
1	1	2
2	1	2
3	2	2
4	1	2

Value counts for 'food_sentiment' (example):

food_sentiment

2 1019023

1 385113

0 121177

Name: count, dtype: int64

Training set size: 1220250

Testing set size: 305063

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarni

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>)

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models and datasets. For more information, see https://huggingface.co/docs/huggingface_hub/guides/auth

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 5.21kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 4.80MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 4.33MB/s]

config.json: 100% 483/483 [00:00<00:00, 61.5kB/s]

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Fal

WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the

model.safetensors: 100% 268M/268M [00:01<00:00, 234MB/s]

Some weights of DistilBertForSequenceClassification were not initialized from the mc
You should probably TRAIN this model on a down-stream task to be able to use it for

Customized DistilBERT model loaded. Output layer size: 18

Using device: cuda

Starting Transformer Model Training...

--- Epoch 1/3 ---

Batch 100/76266, Loss: 0.4605

Batch 200/76266, Loss: 0.5363

Batch 300/76266, Loss: 0.3642

Batch 400/76266, Loss: 0.3450

Batch 500/76266, Loss: 0.4664

Batch 600/76266, Loss: 0.4256
Batch 700/76266, Loss: 0.3760
Batch 800/76266, Loss: 0.3525
Batch 900/76266, Loss: 0.3631
Batch 1000/76266, Loss: 0.3212
Batch 1100/76266, Loss: 0.3153
Batch 1200/76266, Loss: 0.4185
Batch 1300/76266, Loss: 0.3455
Batch 1400/76266, Loss: 0.3565
Batch 1500/76266, Loss: 0.2568
Batch 1600/76266, Loss: 0.3890
Batch 1700/76266, Loss: 0.2809
Batch 1800/76266, Loss: 0.2546
Batch 1900/76266, Loss: 0.2207
Batch 2000/76266, Loss: 0.2464
Batch 2100/76266, Loss: 0.2484
Batch 2200/76266, Loss: 0.3675
Batch 2300/76266, Loss: 0.2434
Batch 2400/76266, Loss: 0.2172
Batch 2500/76266, Loss: 0.3450
Batch 2600/76266, Loss: 0.2391
Batch 2700/76266, Loss: 0.3490
Batch 2800/76266, Loss: 0.2605
Batch 2900/76266, Loss: 0.2697
Batch 3000/76266, Loss: 0.2977
Batch 3100/76266, Loss: 0.2739
Batch 3200/76266, Loss: 0.3967
Batch 3300/76266, Loss: 0.2183
Batch 3400/76266, Loss: 0.1852
Batch 3500/76266, Loss: 0.3476
Batch 3600/76266, Loss: 0.2157
Batch 3700/76266, Loss: 0.3720
Batch 3800/76266, Loss: 0.2292
Batch 3900/76266, Loss: 0.2398
Batch 4000/76266, Loss: 0.2939
Batch 4100/76266, Loss: 0.2868
Batch 4200/76266, Loss: 0.3422
Batch 4300/76266, Loss: 0.2106
Batch 4400/76266, Loss: 0.2370
Batch 4500/76266, Loss: 0.2753
Batch 4600/76266, Loss: 0.2297
Batch 4700/76266, Loss: 0.2656
Batch 4800/76266, Loss: 0.2935
Batch 4900/76266, Loss: 0.2822
Batch 5000/76266, Loss: 0.3437
Batch 5100/76266, Loss: 0.2469
Batch 5200/76266, Loss: 0.2218
Batch 5300/76266, Loss: 0.2201
Batch 5400/76266, Loss: 0.3032
Batch 5500/76266, Loss: 0.2901
Batch 5600/76266, Loss: 0.2854
Batch 5700/76266, Loss: 0.2879
Batch 5800/76266, Loss: 0.2604
Batch 5900/76266, Loss: 0.2035
Batch 6000/76266, Loss: 0.2731
Batch 6100/76266, Loss: 0.3019
Batch 6200/76266, Loss: 0.2162

Batch 6200/76266, Loss: 0.2162
Batch 6300/76266, Loss: 0.1488
Batch 6400/76266, Loss: 0.2188
Batch 6500/76266, Loss: 0.2475
Batch 6600/76266, Loss: 0.3056
Batch 6700/76266, Loss: 0.1772
Batch 6800/76266, Loss: 0.2584
Batch 6900/76266, Loss: 0.2266
Batch 7000/76266, Loss: 0.2180
Batch 7100/76266, Loss: 0.4651
Batch 7200/76266, Loss: 0.3116
Batch 7300/76266, Loss: 0.3559
Batch 7400/76266, Loss: 0.2649
Batch 7500/76266, Loss: 0.2257
Batch 7600/76266, Loss: 0.2698
Batch 7700/76266, Loss: 0.1621
Batch 7800/76266, Loss: 0.2283
Batch 7900/76266, Loss: 0.1570
Batch 8000/76266, Loss: 0.1728
Batch 8100/76266, Loss: 0.2094
Batch 8200/76266, Loss: 0.2318
Batch 8300/76266, Loss: 0.2221
Batch 8400/76266, Loss: 0.2925
Batch 8500/76266, Loss: 0.1719
Batch 8600/76266, Loss: 0.1988
Batch 8700/76266, Loss: 0.2134
Batch 8800/76266, Loss: 0.2269
Batch 8900/76266, Loss: 0.2403
Batch 9000/76266, Loss: 0.2572
Batch 9100/76266, Loss: 0.2907
Batch 9200/76266, Loss: 0.1678
Batch 9300/76266, Loss: 0.2315
Batch 9400/76266, Loss: 0.1683
Batch 9500/76266, Loss: 0.2563
Batch 9600/76266, Loss: 0.1721
Batch 9700/76266, Loss: 0.1617
Batch 9800/76266, Loss: 0.2170
Batch 9900/76266, Loss: 0.2363
Batch 10000/76266, Loss: 0.3079
Batch 10100/76266, Loss: 0.2818
Batch 10200/76266, Loss: 0.2236
Batch 10300/76266, Loss: 0.2009
Batch 10400/76266, Loss: 0.3059
Batch 10500/76266, Loss: 0.3217
Batch 10600/76266, Loss: 0.2569
Batch 10700/76266, Loss: 0.2708
Batch 10800/76266, Loss: 0.3675
Batch 10900/76266, Loss: 0.2785
Batch 11000/76266, Loss: 0.1570
Batch 11100/76266, Loss: 0.1962
Batch 11200/76266, Loss: 0.2333
Batch 11300/76266, Loss: 0.1956
Batch 11400/76266, Loss: 0.3201
Batch 11500/76266, Loss: 0.2251
Batch 11600/76266, Loss: 0.1804
Batch 11700/76266, Loss: 0.2265
Batch 11800/76266, Loss: 0.2019

Batch 11900/76266, Loss: 0.2126
Batch 12000/76266, Loss: 0.3634
Batch 12100/76266, Loss: 0.3363
Batch 12200/76266, Loss: 0.2121
Batch 12300/76266, Loss: 0.2112
Batch 12400/76266, Loss: 0.2091
Batch 12500/76266, Loss: 0.1804
Batch 12600/76266, Loss: 0.2767
Batch 12700/76266, Loss: 0.2527
Batch 12800/76266, Loss: 0.3343
Batch 12900/76266, Loss: 0.2867
Batch 13000/76266, Loss: 0.1403
Batch 13100/76266, Loss: 0.1730
Batch 13200/76266, Loss: 0.1962
Batch 13300/76266, Loss: 0.1862
Batch 13400/76266, Loss: 0.2193
Batch 13500/76266, Loss: 0.2382
Batch 13600/76266, Loss: 0.2506
Batch 13700/76266, Loss: 0.2608
Batch 13800/76266, Loss: 0.3063
Batch 13900/76266, Loss: 0.2544
Batch 14000/76266, Loss: 0.2457
Batch 14100/76266, Loss: 0.1820
Batch 14200/76266, Loss: 0.2121
Batch 14300/76266, Loss: 0.2892
Batch 14400/76266, Loss: 0.2762
Batch 14500/76266, Loss: 0.2478
Batch 14600/76266, Loss: 0.3546
Batch 14700/76266, Loss: 0.2678
Batch 14800/76266, Loss: 0.1930
Batch 14900/76266, Loss: 0.2312
Batch 15000/76266, Loss: 0.2492
Batch 15100/76266, Loss: 0.1468
Batch 15200/76266, Loss: 0.2562
Batch 15300/76266, Loss: 0.2423
Batch 15400/76266, Loss: 0.2149
Batch 15500/76266, Loss: 0.2538
Batch 15600/76266, Loss: 0.2450
Batch 15700/76266, Loss: 0.3074
Batch 15800/76266, Loss: 0.2448
Batch 15900/76266, Loss: 0.2932
Batch 16000/76266, Loss: 0.3259
Batch 16100/76266, Loss: 0.1280
Batch 16200/76266, Loss: 0.2700
Batch 16300/76266, Loss: 0.2143
Batch 16400/76266, Loss: 0.2631
Batch 16500/76266, Loss: 0.1910
Batch 16600/76266, Loss: 0.2024
Batch 16700/76266, Loss: 0.2409
Batch 16800/76266, Loss: 0.2226
Batch 16900/76266, Loss: 0.3664
Batch 17000/76266, Loss: 0.3904
Batch 17100/76266, Loss: 0.2879
Batch 17200/76266, Loss: 0.1900
Batch 17300/76266, Loss: 0.3143
Batch 17400/76266, Loss: 0.3596
Batch 17500/76266, Loss: 0.3868

Batch 17500/76266, Loss: 0.2000
Batch 17600/76266, Loss: 0.3185
Batch 17700/76266, Loss: 0.3433
Batch 17800/76266, Loss: 0.1642
Batch 17900/76266, Loss: 0.2780
Batch 18000/76266, Loss: 0.2497
Batch 18100/76266, Loss: 0.3029
Batch 18200/76266, Loss: 0.2133
Batch 18300/76266, Loss: 0.2538
Batch 18400/76266, Loss: 0.1484
Batch 18500/76266, Loss: 0.2488
Batch 18600/76266, Loss: 0.2301
Batch 18700/76266, Loss: 0.2111
Batch 18800/76266, Loss: 0.2983
Batch 18900/76266, Loss: 0.2572
Batch 19000/76266, Loss: 0.1671
Batch 19100/76266, Loss: 0.2175
Batch 19200/76266, Loss: 0.2349
Batch 19300/76266, Loss: 0.3254
Batch 19400/76266, Loss: 0.3029
Batch 19500/76266, Loss: 0.2364
Batch 19600/76266, Loss: 0.2604
Batch 19700/76266, Loss: 0.1362
Batch 19800/76266, Loss: 0.3143
Batch 19900/76266, Loss: 0.3204
Batch 20000/76266, Loss: 0.2638
Batch 20100/76266, Loss: 0.2057
Batch 20200/76266, Loss: 0.2447
Batch 20300/76266, Loss: 0.1646
Batch 20400/76266, Loss: 0.2188
Batch 20500/76266, Loss: 0.1546
Batch 20600/76266, Loss: 0.3031
Batch 20700/76266, Loss: 0.2330
Batch 20800/76266, Loss: 0.2741
Batch 20900/76266, Loss: 0.2615
Batch 21000/76266, Loss: 0.2846
Batch 21100/76266, Loss: 0.1039
Batch 21200/76266, Loss: 0.1959
Batch 21300/76266, Loss: 0.2193
Batch 21400/76266, Loss: 0.2199
Batch 21500/76266, Loss: 0.2410
Batch 21600/76266, Loss: 0.1617
Batch 21700/76266, Loss: 0.2123
Batch 21800/76266, Loss: 0.2898
Batch 21900/76266, Loss: 0.2996
Batch 22000/76266, Loss: 0.1674
Batch 22100/76266, Loss: 0.1422
Batch 22200/76266, Loss: 0.1731
Batch 22300/76266, Loss: 0.2551
Batch 22400/76266, Loss: 0.3240
Batch 22500/76266, Loss: 0.2284
Batch 22600/76266, Loss: 0.3538
Batch 22700/76266, Loss: 0.1725
Batch 22800/76266, Loss: 0.1338
Batch 22900/76266, Loss: 0.1635
Batch 23000/76266, Loss: 0.3660
Batch 23100/76266, Loss: 0.3493

Batch 23200/76266, Loss: 0.2593
Batch 23300/76266, Loss: 0.2686
Batch 23400/76266, Loss: 0.3398
Batch 23500/76266, Loss: 0.1725
Batch 23600/76266, Loss: 0.2390
Batch 23700/76266, Loss: 0.2645
Batch 23800/76266, Loss: 0.1634
Batch 23900/76266, Loss: 0.2515
Batch 24000/76266, Loss: 0.1863
Batch 24100/76266, Loss: 0.2096
Batch 24200/76266, Loss: 0.2190
Batch 24300/76266, Loss: 0.2342
Batch 24400/76266, Loss: 0.1903
Batch 24500/76266, Loss: 0.2107
Batch 24600/76266, Loss: 0.2453
Batch 24700/76266, Loss: 0.3048
Batch 24800/76266, Loss: 0.1693
Batch 24900/76266, Loss: 0.1019
Batch 25000/76266, Loss: 0.2731
Batch 25100/76266, Loss: 0.2476
Batch 25200/76266, Loss: 0.2740
Batch 25300/76266, Loss: 0.2234
Batch 25400/76266, Loss: 0.2749
Batch 25500/76266, Loss: 0.3239
Batch 25600/76266, Loss: 0.1759
Batch 25700/76266, Loss: 0.1550
Batch 25800/76266, Loss: 0.3333
Batch 25900/76266, Loss: 0.2930
Batch 26000/76266, Loss: 0.2031
Batch 26100/76266, Loss: 0.2615
Batch 26200/76266, Loss: 0.1789
Batch 26300/76266, Loss: 0.1641
Batch 26400/76266, Loss: 0.2188
Batch 26500/76266, Loss: 0.2428
Batch 26600/76266, Loss: 0.2549
Batch 26700/76266, Loss: 0.2244
Batch 26800/76266, Loss: 0.2534
Batch 26900/76266, Loss: 0.1456
Batch 27000/76266, Loss: 0.1602
Batch 27100/76266, Loss: 0.2574
Batch 27200/76266, Loss: 0.2382
Batch 27300/76266, Loss: 0.2180
Batch 27400/76266, Loss: 0.2230
Batch 27500/76266, Loss: 0.2046
Batch 27600/76266, Loss: 0.3032
Batch 27700/76266, Loss: 0.1903
Batch 27800/76266, Loss: 0.2648
Batch 27900/76266, Loss: 0.1562
Batch 28000/76266, Loss: 0.2593
Batch 28100/76266, Loss: 0.2646
Batch 28200/76266, Loss: 0.1697
Batch 28300/76266, Loss: 0.2197
Batch 28400/76266, Loss: 0.2383
Batch 28500/76266, Loss: 0.2706
Batch 28600/76266, Loss: 0.2225
Batch 28700/76266, Loss: 0.2846
Batch 28800/76266, Loss: 0.2620

Batch 28800/76266, Loss: 0.2020
Batch 28900/76266, Loss: 0.2307
Batch 29000/76266, Loss: 0.1964
Batch 29100/76266, Loss: 0.1291
Batch 29200/76266, Loss: 0.2664
Batch 29300/76266, Loss: 0.2709
Batch 29400/76266, Loss: 0.2695
Batch 29500/76266, Loss: 0.1597
Batch 29600/76266, Loss: 0.3364
Batch 29700/76266, Loss: 0.2313
Batch 29800/76266, Loss: 0.1808
Batch 29900/76266, Loss: 0.3994
Batch 30000/76266, Loss: 0.1997
Batch 30100/76266, Loss: 0.1902
Batch 30200/76266, Loss: 0.1853
Batch 30300/76266, Loss: 0.1439
Batch 30400/76266, Loss: 0.2629
Batch 30500/76266, Loss: 0.1843
Batch 30600/76266, Loss: 0.1899
Batch 30700/76266, Loss: 0.1812
Batch 30800/76266, Loss: 0.2181
Batch 30900/76266, Loss: 0.1917
Batch 31000/76266, Loss: 0.1754
Batch 31100/76266, Loss: 0.2329
Batch 31200/76266, Loss: 0.1662
Batch 31300/76266, Loss: 0.2347
Batch 31400/76266, Loss: 0.2663
Batch 31500/76266, Loss: 0.1844
Batch 31600/76266, Loss: 0.1455
Batch 31700/76266, Loss: 0.1703
Batch 31800/76266, Loss: 0.1697
Batch 31900/76266, Loss: 0.1565
Batch 32000/76266, Loss: 0.2556
Batch 32100/76266, Loss: 0.3518
Batch 32200/76266, Loss: 0.3054
Batch 32300/76266, Loss: 0.1779
Batch 32400/76266, Loss: 0.2076
Batch 32500/76266, Loss: 0.2088
Batch 32600/76266, Loss: 0.2078
Batch 32700/76266, Loss: 0.2368
Batch 32800/76266, Loss: 0.1786
Batch 32900/76266, Loss: 0.2044
Batch 33000/76266, Loss: 0.1760
Batch 33100/76266, Loss: 0.2892
Batch 33200/76266, Loss: 0.2601
Batch 33300/76266, Loss: 0.1980
Batch 33400/76266, Loss: 0.1340
Batch 33500/76266, Loss: 0.3303
Batch 33600/76266, Loss: 0.2630
Batch 33700/76266, Loss: 0.2218
Batch 33800/76266, Loss: 0.2351
Batch 33900/76266, Loss: 0.2205
Batch 34000/76266, Loss: 0.2216
Batch 34100/76266, Loss: 0.2219
Batch 34200/76266, Loss: 0.1180
Batch 34300/76266, Loss: 0.2250
Batch 34400/76266, Loss: 0.1894

Batch 34500/76266, Loss: 0.2536
Batch 34600/76266, Loss: 0.3357
Batch 34700/76266, Loss: 0.2658
Batch 34800/76266, Loss: 0.1544
Batch 34900/76266, Loss: 0.2647
Batch 35000/76266, Loss: 0.1970
Batch 35100/76266, Loss: 0.1673
Batch 35200/76266, Loss: 0.2063
Batch 35300/76266, Loss: 0.2289
Batch 35400/76266, Loss: 0.2131
Batch 35500/76266, Loss: 0.2119
Batch 35600/76266, Loss: 0.3056
Batch 35700/76266, Loss: 0.1717
Batch 35800/76266, Loss: 0.2051
Batch 35900/76266, Loss: 0.2968
Batch 36000/76266, Loss: 0.2155
Batch 36100/76266, Loss: 0.1978
Batch 36200/76266, Loss: 0.2441
Batch 36300/76266, Loss: 0.1694
Batch 36400/76266, Loss: 0.2874
Batch 36500/76266, Loss: 0.3214
Batch 36600/76266, Loss: 0.1707
Batch 36700/76266, Loss: 0.2490
Batch 36800/76266, Loss: 0.2530
Batch 36900/76266, Loss: 0.2173
Batch 37000/76266, Loss: 0.1928
Batch 37100/76266, Loss: 0.1913
Batch 37200/76266, Loss: 0.1842
Batch 37300/76266, Loss: 0.2607
Batch 37400/76266, Loss: 0.2394
Batch 37500/76266, Loss: 0.2119
Batch 37600/76266, Loss: 0.1837
Batch 37700/76266, Loss: 0.1526
Batch 37800/76266, Loss: 0.2440
Batch 37900/76266, Loss: 0.2323
Batch 38000/76266, Loss: 0.3739
Batch 38100/76266, Loss: 0.1430
Batch 38200/76266, Loss: 0.1720
Batch 38300/76266, Loss: 0.2033
Batch 38400/76266, Loss: 0.1741
Batch 38500/76266, Loss: 0.1850
Batch 38600/76266, Loss: 0.3159
Batch 38700/76266, Loss: 0.2704
Batch 38800/76266, Loss: 0.2097
Batch 38900/76266, Loss: 0.1351
Batch 39000/76266, Loss: 0.2041
Batch 39100/76266, Loss: 0.1652
Batch 39200/76266, Loss: 0.3000
Batch 39300/76266, Loss: 0.2107
Batch 39400/76266, Loss: 0.2053
Batch 39500/76266, Loss: 0.1944
Batch 39600/76266, Loss: 0.1800
Batch 39700/76266, Loss: 0.2093
Batch 39800/76266, Loss: 0.2186
Batch 39900/76266, Loss: 0.3000
Batch 40000/76266, Loss: 0.2496
Batch 40100/76266, Loss: 0.2581

Batch 40100/76266, Loss: 0.2301
Batch 40200/76266, Loss: 0.2052
Batch 40300/76266, Loss: 0.2578
Batch 40400/76266, Loss: 0.1791
Batch 40500/76266, Loss: 0.2315
Batch 40600/76266, Loss: 0.2474
Batch 40700/76266, Loss: 0.2373
Batch 40800/76266, Loss: 0.3760
Batch 40900/76266, Loss: 0.2230
Batch 41000/76266, Loss: 0.2254
Batch 41100/76266, Loss: 0.1642
Batch 41200/76266, Loss: 0.1571
Batch 41300/76266, Loss: 0.2433
Batch 41400/76266, Loss: 0.1987
Batch 41500/76266, Loss: 0.3521
Batch 41600/76266, Loss: 0.1458
Batch 41700/76266, Loss: 0.1839
Batch 41800/76266, Loss: 0.2057
Batch 41900/76266, Loss: 0.1532
Batch 42000/76266, Loss: 0.2379
Batch 42100/76266, Loss: 0.2749
Batch 42200/76266, Loss: 0.3214
Batch 42300/76266, Loss: 0.2278
Batch 42400/76266, Loss: 0.1708
Batch 42500/76266, Loss: 0.3077
Batch 42600/76266, Loss: 0.1744
Batch 42700/76266, Loss: 0.1982
Batch 42800/76266, Loss: 0.2547
Batch 42900/76266, Loss: 0.1171
Batch 43000/76266, Loss: 0.2412
Batch 43100/76266, Loss: 0.2161
Batch 43200/76266, Loss: 0.2359
Batch 43300/76266, Loss: 0.2553
Batch 43400/76266, Loss: 0.3019
Batch 43500/76266, Loss: 0.3436
Batch 43600/76266, Loss: 0.2771
Batch 43700/76266, Loss: 0.3974
Batch 43800/76266, Loss: 0.2414
Batch 43900/76266, Loss: 0.2838
Batch 44000/76266, Loss: 0.2686
Batch 44100/76266, Loss: 0.3237
Batch 44200/76266, Loss: 0.1895
Batch 44300/76266, Loss: 0.1920
Batch 44400/76266, Loss: 0.2363
Batch 44500/76266, Loss: 0.2555
Batch 44600/76266, Loss: 0.2397
Batch 44700/76266, Loss: 0.3245
Batch 44800/76266, Loss: 0.2605
Batch 44900/76266, Loss: 0.3423
Batch 45000/76266, Loss: 0.1884
Batch 45100/76266, Loss: 0.2218
Batch 45200/76266, Loss: 0.1843
Batch 45300/76266, Loss: 0.4388
Batch 45400/76266, Loss: 0.2182
Batch 45500/76266, Loss: 0.1500
Batch 45600/76266, Loss: 0.1256
Batch 45700/76266, Loss: 0.2064

Batch 45800/76266, Loss: 0.2118
Batch 45900/76266, Loss: 0.1950
Batch 46000/76266, Loss: 0.1432
Batch 46100/76266, Loss: 0.1841
Batch 46200/76266, Loss: 0.2502
Batch 46300/76266, Loss: 0.2584
Batch 46400/76266, Loss: 0.2765
Batch 46500/76266, Loss: 0.2676
Batch 46600/76266, Loss: 0.2361
Batch 46700/76266, Loss: 0.1274
Batch 46800/76266, Loss: 0.2227
Batch 46900/76266, Loss: 0.1791
Batch 47000/76266, Loss: 0.2441
Batch 47100/76266, Loss: 0.3046
Batch 47200/76266, Loss: 0.2591
Batch 47300/76266, Loss: 0.2721
Batch 47400/76266, Loss: 0.2244
Batch 47500/76266, Loss: 0.1832
Batch 47600/76266, Loss: 0.1480
Batch 47700/76266, Loss: 0.1917
Batch 47800/76266, Loss: 0.1436
Batch 47900/76266, Loss: 0.1629
Batch 48000/76266, Loss: 0.1779
Batch 48100/76266, Loss: 0.1897
Batch 48200/76266, Loss: 0.2579
Batch 48300/76266, Loss: 0.2090
Batch 48400/76266, Loss: 0.1793
Batch 48500/76266, Loss: 0.2478
Batch 48600/76266, Loss: 0.2246
Batch 48700/76266, Loss: 0.2450
Batch 48800/76266, Loss: 0.2844
Batch 48900/76266, Loss: 0.2988
Batch 49000/76266, Loss: 0.2079
Batch 49100/76266, Loss: 0.1269
Batch 49200/76266, Loss: 0.3211
Batch 49300/76266, Loss: 0.1556
Batch 49400/76266, Loss: 0.2183
Batch 49500/76266, Loss: 0.2133
Batch 49600/76266, Loss: 0.2655
Batch 49700/76266, Loss: 0.1822
Batch 49800/76266, Loss: 0.1756
Batch 49900/76266, Loss: 0.1956
Batch 50000/76266, Loss: 0.1960
Batch 50100/76266, Loss: 0.1705
Batch 50200/76266, Loss: 0.1545
Batch 50300/76266, Loss: 0.2453
Batch 50400/76266, Loss: 0.2726
Batch 50500/76266, Loss: 0.2563
Batch 50600/76266, Loss: 0.3628
Batch 50700/76266, Loss: 0.1704
Batch 50800/76266, Loss: 0.2076
Batch 50900/76266, Loss: 0.2933
Batch 51000/76266, Loss: 0.2966
Batch 51100/76266, Loss: 0.2126
Batch 51200/76266, Loss: 0.1901
Batch 51300/76266, Loss: 0.2749
Batch 51400/76266, Loss: 0.1606

Batch 51400/76266, Loss: 0.1050
Batch 51500/76266, Loss: 0.2667
Batch 51600/76266, Loss: 0.2024
Batch 51700/76266, Loss: 0.1844
Batch 51800/76266, Loss: 0.2493
Batch 51900/76266, Loss: 0.2170
Batch 52000/76266, Loss: 0.1698
Batch 52100/76266, Loss: 0.1564
Batch 52200/76266, Loss: 0.2628
Batch 52300/76266, Loss: 0.1513
Batch 52400/76266, Loss: 0.1694
Batch 52500/76266, Loss: 0.1645
Batch 52600/76266, Loss: 0.2374
Batch 52700/76266, Loss: 0.2077
Batch 52800/76266, Loss: 0.1593
Batch 52900/76266, Loss: 0.2849
Batch 53000/76266, Loss: 0.2284
Batch 53100/76266, Loss: 0.2887
Batch 53200/76266, Loss: 0.1646
Batch 53300/76266, Loss: 0.2694
Batch 53400/76266, Loss: 0.1619
Batch 53500/76266, Loss: 0.2009
Batch 53600/76266, Loss: 0.3400
Batch 53700/76266, Loss: 0.1866
Batch 53800/76266, Loss: 0.1925
Batch 53900/76266, Loss: 0.2021
Batch 54000/76266, Loss: 0.2504
Batch 54100/76266, Loss: 0.2272
Batch 54200/76266, Loss: 0.2211
Batch 54300/76266, Loss: 0.2401
Batch 54400/76266, Loss: 0.1966
Batch 54500/76266, Loss: 0.2236
Batch 54600/76266, Loss: 0.2507
Batch 54700/76266, Loss: 0.1529
Batch 54800/76266, Loss: 0.2310
Batch 54900/76266, Loss: 0.1411
Batch 55000/76266, Loss: 0.1300
Batch 55100/76266, Loss: 0.1752
Batch 55200/76266, Loss: 0.2332
Batch 55300/76266, Loss: 0.2599
Batch 55400/76266, Loss: 0.2622
Batch 55500/76266, Loss: 0.1708
Batch 55600/76266, Loss: 0.2637
Batch 55700/76266, Loss: 0.1459
Batch 55800/76266, Loss: 0.1663
Batch 55900/76266, Loss: 0.2879
Batch 56000/76266, Loss: 0.1696
Batch 56100/76266, Loss: 0.1583
Batch 56200/76266, Loss: 0.2916
Batch 56300/76266, Loss: 0.3343
Batch 56400/76266, Loss: 0.2248
Batch 56500/76266, Loss: 0.2924
Batch 56600/76266, Loss: 0.2609
Batch 56700/76266, Loss: 0.2299
Batch 56800/76266, Loss: 0.1292
Batch 56900/76266, Loss: 0.3522
Batch 57000/76266, Loss: 0.1667

Batch 57100/76266, Loss: 0.1437
Batch 57200/76266, Loss: 0.1917
Batch 57300/76266, Loss: 0.3051
Batch 57400/76266, Loss: 0.2060
Batch 57500/76266, Loss: 0.2232
Batch 57600/76266, Loss: 0.1349
Batch 57700/76266, Loss: 0.1616
Batch 57800/76266, Loss: 0.2380
Batch 57900/76266, Loss: 0.1418
Batch 58000/76266, Loss: 0.2970
Batch 58100/76266, Loss: 0.1640
Batch 58200/76266, Loss: 0.2101
Batch 58300/76266, Loss: 0.1510
Batch 58400/76266, Loss: 0.1852
Batch 58500/76266, Loss: 0.2241
Batch 58600/76266, Loss: 0.2003
Batch 58700/76266, Loss: 0.0806
Batch 58800/76266, Loss: 0.1848
Batch 58900/76266, Loss: 0.1699
Batch 59000/76266, Loss: 0.1754
Batch 59100/76266, Loss: 0.1238
Batch 59200/76266, Loss: 0.1460
Batch 59300/76266, Loss: 0.2125
Batch 59400/76266, Loss: 0.3222
Batch 59500/76266, Loss: 0.2316
Batch 59600/76266, Loss: 0.1590
Batch 59700/76266, Loss: 0.2831
Batch 59800/76266, Loss: 0.1840
Batch 59900/76266, Loss: 0.2020
Batch 60000/76266, Loss: 0.1827
Batch 60100/76266, Loss: 0.2069
Batch 60200/76266, Loss: 0.1930
Batch 60300/76266, Loss: 0.1921
Batch 60400/76266, Loss: 0.2156
Batch 60500/76266, Loss: 0.1187
Batch 60600/76266, Loss: 0.1712
Batch 60700/76266, Loss: 0.3552
Batch 60800/76266, Loss: 0.2710
Batch 60900/76266, Loss: 0.2756
Batch 61000/76266, Loss: 0.1536
Batch 61100/76266, Loss: 0.1521
Batch 61200/76266, Loss: 0.2935
Batch 61300/76266, Loss: 0.1818
Batch 61400/76266, Loss: 0.2181
Batch 61500/76266, Loss: 0.2282
Batch 61600/76266, Loss: 0.1758
Batch 61700/76266, Loss: 0.1810
Batch 61800/76266, Loss: 0.1330
Batch 61900/76266, Loss: 0.2060
Batch 62000/76266, Loss: 0.1277
Batch 62100/76266, Loss: 0.1955
Batch 62200/76266, Loss: 0.1409
Batch 62300/76266, Loss: 0.2589
Batch 62400/76266, Loss: 0.2233
Batch 62500/76266, Loss: 0.1626
Batch 62600/76266, Loss: 0.1778
Batch 62700/76266, Loss: 0.2750

Batch 62700/76266, Loss: 0.2133
Batch 62800/76266, Loss: 0.2163
Batch 62900/76266, Loss: 0.2587
Batch 63000/76266, Loss: 0.1605
Batch 63100/76266, Loss: 0.3236
Batch 63200/76266, Loss: 0.1869
Batch 63300/76266, Loss: 0.1877
Batch 63400/76266, Loss: 0.1517
Batch 63500/76266, Loss: 0.1382
Batch 63600/76266, Loss: 0.1479
Batch 63700/76266, Loss: 0.2410
Batch 63800/76266, Loss: 0.3592
Batch 63900/76266, Loss: 0.2405
Batch 64000/76266, Loss: 0.2050
Batch 64100/76266, Loss: 0.2510
Batch 64200/76266, Loss: 0.2070
Batch 64300/76266, Loss: 0.2429
Batch 64400/76266, Loss: 0.2329
Batch 64500/76266, Loss: 0.1103
Batch 64600/76266, Loss: 0.3622
Batch 64700/76266, Loss: 0.2543
Batch 64800/76266, Loss: 0.2452
Batch 64900/76266, Loss: 0.1409
Batch 65000/76266, Loss: 0.1544
Batch 65100/76266, Loss: 0.1918
Batch 65200/76266, Loss: 0.2413
Batch 65300/76266, Loss: 0.2059
Batch 65400/76266, Loss: 0.2515
Batch 65500/76266, Loss: 0.3064
Batch 65600/76266, Loss: 0.2646
Batch 65700/76266, Loss: 0.1764
Batch 65800/76266, Loss: 0.2055
Batch 65900/76266, Loss: 0.1454
Batch 66000/76266, Loss: 0.1959
Batch 66100/76266, Loss: 0.2153
Batch 66200/76266, Loss: 0.1484
Batch 66300/76266, Loss: 0.2316
Batch 66400/76266, Loss: 0.2763
Batch 66500/76266, Loss: 0.3087
Batch 66600/76266, Loss: 0.3112
Batch 66700/76266, Loss: 0.2038
Batch 66800/76266, Loss: 0.1166
Batch 66900/76266, Loss: 0.0851
Batch 67000/76266, Loss: 0.2649
Batch 67100/76266, Loss: 0.1900
Batch 67200/76266, Loss: 0.1883
Batch 67300/76266, Loss: 0.1823
Batch 67400/76266, Loss: 0.1665
Batch 67500/76266, Loss: 0.3330
Batch 67600/76266, Loss: 0.1630
Batch 67700/76266, Loss: 0.1917
Batch 67800/76266, Loss: 0.2246
Batch 67900/76266, Loss: 0.1984
Batch 68000/76266, Loss: 0.1862
Batch 68100/76266, Loss: 0.2166
Batch 68200/76266, Loss: 0.2538
Batch 68300/76266, Loss: 0.1941

Batch 68400/76266, Loss: 0.2561
Batch 68500/76266, Loss: 0.3024
Batch 68600/76266, Loss: 0.1831
Batch 68700/76266, Loss: 0.3283
Batch 68800/76266, Loss: 0.3222
Batch 68900/76266, Loss: 0.2339
Batch 69000/76266, Loss: 0.1715
Batch 69100/76266, Loss: 0.2545
Batch 69200/76266, Loss: 0.2288
Batch 69300/76266, Loss: 0.2864
Batch 69400/76266, Loss: 0.1956
Batch 69500/76266, Loss: 0.1601
Batch 69600/76266, Loss: 0.2863
Batch 69700/76266, Loss: 0.2075
Batch 69800/76266, Loss: 0.1826
Batch 69900/76266, Loss: 0.2274
Batch 70000/76266, Loss: 0.2315
Batch 70100/76266, Loss: 0.1926
Batch 70200/76266, Loss: 0.1881
Batch 70300/76266, Loss: 0.1406
Batch 70400/76266, Loss: 0.2825
Batch 70500/76266, Loss: 0.3090
Batch 70600/76266, Loss: 0.1973
Batch 70700/76266, Loss: 0.2346
Batch 70800/76266, Loss: 0.1468
Batch 70900/76266, Loss: 0.2899
Batch 71000/76266, Loss: 0.1943
Batch 71100/76266, Loss: 0.3047
Batch 71200/76266, Loss: 0.2230
Batch 71300/76266, Loss: 0.3505
Batch 71400/76266, Loss: 0.1546
Batch 71500/76266, Loss: 0.1388
Batch 71600/76266, Loss: 0.3901
Batch 71700/76266, Loss: 0.3550
Batch 71800/76266, Loss: 0.2159
Batch 71900/76266, Loss: 0.2480
Batch 72000/76266, Loss: 0.1412
Batch 72100/76266, Loss: 0.2043
Batch 72200/76266, Loss: 0.2433
Batch 72300/76266, Loss: 0.2576
Batch 72400/76266, Loss: 0.2723
Batch 72500/76266, Loss: 0.2625
Batch 72600/76266, Loss: 0.1977
Batch 72700/76266, Loss: 0.2607
Batch 72800/76266, Loss: 0.1929
Batch 72900/76266, Loss: 0.1850
Batch 73000/76266, Loss: 0.2521
Batch 73100/76266, Loss: 0.2744
Batch 73200/76266, Loss: 0.2139
Batch 73300/76266, Loss: 0.1946
Batch 73400/76266, Loss: 0.2176
Batch 73500/76266, Loss: 0.1423
Batch 73600/76266, Loss: 0.2555
Batch 73700/76266, Loss: 0.2724
Batch 73800/76266, Loss: 0.2304
Batch 73900/76266, Loss: 0.1860
Batch 74000/76266, Loss: 0.2505

```
Batch 74000/76266, Loss: 0.2333
Batch 74100/76266, Loss: 0.4019
Batch 74200/76266, Loss: 0.2285
Batch 74300/76266, Loss: 0.2496
Batch 74400/76266, Loss: 0.2865
Batch 74500/76266, Loss: 0.2436
Batch 74600/76266, Loss: 0.1338
Batch 74700/76266, Loss: 0.2227
Batch 74800/76266, Loss: 0.2339
Batch 74900/76266, Loss: 0.1909
Batch 75000/76266, Loss: 0.2297
Batch 75100/76266, Loss: 0.1944
Batch 75200/76266, Loss: 0.4000
Batch 75300/76266, Loss: 0.2231
Batch 75400/76266, Loss: 0.2479
Batch 75500/76266, Loss: 0.2480
Batch 75600/76266, Loss: 0.2889
Batch 75700/76266, Loss: 0.0870
Batch 75800/76266, Loss: 0.2655
Batch 75900/76266, Loss: 0.2343
Batch 76000/76266, Loss: 0.2004
Batch 76100/76266, Loss: 0.2114
Batch 76200/76266, Loss: 0.2609
```

Epoch 1 completed in 11584.55s. Average Training Loss: 0.2328

--- Epoch 2/3 ---

```
Batch 100/76266, Loss: 0.2105
Batch 200/76266, Loss: 0.1638
Batch 300/76266, Loss: 0.1383
Batch 400/76266, Loss: 0.2674
Batch 500/76266, Loss: 0.2163
Batch 600/76266, Loss: 0.1804
Batch 700/76266, Loss: 0.1603
Batch 800/76266, Loss: 0.2189
Batch 900/76266, Loss: 0.1098
Batch 1000/76266, Loss: 0.1440
Batch 1100/76266, Loss: 0.1603
Batch 1200/76266, Loss: 0.1470
Batch 1300/76266, Loss: 0.1641
Batch 1400/76266, Loss: 0.1391
Batch 1500/76266, Loss: 0.2448
Batch 1600/76266, Loss: 0.1885
Batch 1700/76266, Loss: 0.1828
Batch 1800/76266, Loss: 0.2410
Batch 1900/76266, Loss: 0.1717
Batch 2000/76266, Loss: 0.3174
Batch 2100/76266, Loss: 0.2821
Batch 2200/76266, Loss: 0.1580
Batch 2300/76266, Loss: 0.2170
Batch 2400/76266, Loss: 0.2001
Batch 2500/76266, Loss: 0.1504
Batch 2600/76266, Loss: 0.2113
Batch 2700/76266, Loss: 0.2114
Batch 2800/76266, Loss: 0.2728
Batch 2900/76266, Loss: 0.2267
Batch 3000/76266, Loss: 0.1386
Batch 3100/76266, Loss: 0.2207
Batch 3200/76266, Loss: 0.1674
```

Batch 3300/76266, Loss: 0.1242
Batch 3400/76266, Loss: 0.1989
Batch 3500/76266, Loss: 0.1775
Batch 3600/76266, Loss: 0.1760
Batch 3700/76266, Loss: 0.1793
Batch 3800/76266, Loss: 0.1873
Batch 3900/76266, Loss: 0.1157
Batch 4000/76266, Loss: 0.1409
Batch 4100/76266, Loss: 0.1648
Batch 4200/76266, Loss: 0.2395
Batch 4300/76266, Loss: 0.0860
Batch 4400/76266, Loss: 0.1292
Batch 4500/76266, Loss: 0.1498
Batch 4600/76266, Loss: 0.2661
Batch 4700/76266, Loss: 0.1968
Batch 4800/76266, Loss: 0.1737
Batch 4900/76266, Loss: 0.2460
Batch 5000/76266, Loss: 0.3175
Batch 5100/76266, Loss: 0.2355
Batch 5200/76266, Loss: 0.2513
Batch 5300/76266, Loss: 0.2773
Batch 5400/76266, Loss: 0.1058
Batch 5500/76266, Loss: 0.1701
Batch 5600/76266, Loss: 0.2258
Batch 5700/76266, Loss: 0.1250
Batch 5800/76266, Loss: 0.1784
Batch 5900/76266, Loss: 0.2783
Batch 6000/76266, Loss: 0.1783
Batch 6100/76266, Loss: 0.1183
Batch 6200/76266, Loss: 0.2509
Batch 6300/76266, Loss: 0.1793
Batch 6400/76266, Loss: 0.3047
Batch 6500/76266, Loss: 0.2407
Batch 6600/76266, Loss: 0.1991
Batch 6700/76266, Loss: 0.3328
Batch 6800/76266, Loss: 0.2172
Batch 6900/76266, Loss: 0.1514
Batch 7000/76266, Loss: 0.1852
Batch 7100/76266, Loss: 0.2463
Batch 7200/76266, Loss: 0.1524
Batch 7300/76266, Loss: 0.1532
Batch 7400/76266, Loss: 0.2208
Batch 7500/76266, Loss: 0.1678
Batch 7600/76266, Loss: 0.1563
Batch 7700/76266, Loss: 0.3154
Batch 7800/76266, Loss: 0.1801
Batch 7900/76266, Loss: 0.2704
Batch 8000/76266, Loss: 0.2629
Batch 8100/76266, Loss: 0.1520
Batch 8200/76266, Loss: 0.2579
Batch 8300/76266, Loss: 0.2550
Batch 8400/76266, Loss: 0.1405
Batch 8500/76266, Loss: 0.2177
Batch 8600/76266, Loss: 0.2155
Batch 8700/76266, Loss: 0.1693
Batch 8800/76266, Loss: 0.1919
Batch 8900/76266, Loss: 0.1969

Batch 8900/76266, Loss: 0.1505
Batch 9000/76266, Loss: 0.1651
Batch 9100/76266, Loss: 0.1979
Batch 9200/76266, Loss: 0.1173
Batch 9300/76266, Loss: 0.1816
Batch 9400/76266, Loss: 0.1899
Batch 9500/76266, Loss: 0.1890
Batch 9600/76266, Loss: 0.1568
Batch 9700/76266, Loss: 0.1428
Batch 9800/76266, Loss: 0.2692
Batch 9900/76266, Loss: 0.1716
Batch 10000/76266, Loss: 0.3178
Batch 10100/76266, Loss: 0.2166
Batch 10200/76266, Loss: 0.3076
Batch 10300/76266, Loss: 0.2212
Batch 10400/76266, Loss: 0.2583
Batch 10500/76266, Loss: 0.2664
Batch 10600/76266, Loss: 0.2343
Batch 10700/76266, Loss: 0.1795
Batch 10800/76266, Loss: 0.1917
Batch 10900/76266, Loss: 0.1582
Batch 11000/76266, Loss: 0.1316
Batch 11100/76266, Loss: 0.2354
Batch 11200/76266, Loss: 0.2117
Batch 11300/76266, Loss: 0.3327
Batch 11400/76266, Loss: 0.1552
Batch 11500/76266, Loss: 0.1102
Batch 11600/76266, Loss: 0.2219
Batch 11700/76266, Loss: 0.1993
Batch 11800/76266, Loss: 0.1182
Batch 11900/76266, Loss: 0.2281
Batch 12000/76266, Loss: 0.2000
Batch 12100/76266, Loss: 0.1779
Batch 12200/76266, Loss: 0.3171
Batch 12300/76266, Loss: 0.1878
Batch 12400/76266, Loss: 0.2206
Batch 12500/76266, Loss: 0.2907
Batch 12600/76266, Loss: 0.1576
Batch 12700/76266, Loss: 0.1692
Batch 12800/76266, Loss: 0.1554
Batch 12900/76266, Loss: 0.2148
Batch 13000/76266, Loss: 0.1851
Batch 13100/76266, Loss: 0.2216
Batch 13200/76266, Loss: 0.1697
Batch 13300/76266, Loss: 0.2610
Batch 13400/76266, Loss: 0.1804
Batch 13500/76266, Loss: 0.1653
Batch 13600/76266, Loss: 0.2483
Batch 13700/76266, Loss: 0.1947
Batch 13800/76266, Loss: 0.2559
Batch 13900/76266, Loss: 0.2465
Batch 14000/76266, Loss: 0.1104
Batch 14100/76266, Loss: 0.1971
Batch 14200/76266, Loss: 0.1670
Batch 14300/76266, Loss: 0.2024
Batch 14400/76266, Loss: 0.3280
Batch 14500/76266, Loss: 0.1854

Batch 14600/76266, Loss: 0.2117
Batch 14700/76266, Loss: 0.1607
Batch 14800/76266, Loss: 0.1986
Batch 14900/76266, Loss: 0.2530
Batch 15000/76266, Loss: 0.2417
Batch 15100/76266, Loss: 0.2722
Batch 15200/76266, Loss: 0.2610
Batch 15300/76266, Loss: 0.2392
Batch 15400/76266, Loss: 0.2615
Batch 15500/76266, Loss: 0.2020
Batch 15600/76266, Loss: 0.1852
Batch 15700/76266, Loss: 0.1875
Batch 15800/76266, Loss: 0.1880
Batch 15900/76266, Loss: 0.1763
Batch 16000/76266, Loss: 0.1617
Batch 16100/76266, Loss: 0.2646
Batch 16200/76266, Loss: 0.1399
Batch 16300/76266, Loss: 0.2942
Batch 16400/76266, Loss: 0.2131
Batch 16500/76266, Loss: 0.2222
Batch 16600/76266, Loss: 0.1747
Batch 16700/76266, Loss: 0.2372
Batch 16800/76266, Loss: 0.2440
Batch 16900/76266, Loss: 0.1493
Batch 17000/76266, Loss: 0.1526
Batch 17100/76266, Loss: 0.1760
Batch 17200/76266, Loss: 0.0980
Batch 17300/76266, Loss: 0.1767
Batch 17400/76266, Loss: 0.1968
Batch 17500/76266, Loss: 0.2328
Batch 17600/76266, Loss: 0.2267
Batch 17700/76266, Loss: 0.1671
Batch 17800/76266, Loss: 0.1415
Batch 17900/76266, Loss: 0.2599
Batch 18000/76266, Loss: 0.2005
Batch 18100/76266, Loss: 0.1937
Batch 18200/76266, Loss: 0.1842
Batch 18300/76266, Loss: 0.1513
Batch 18400/76266, Loss: 0.1465
Batch 18500/76266, Loss: 0.1867
Batch 18600/76266, Loss: 0.1580
Batch 18700/76266, Loss: 0.3194
Batch 18800/76266, Loss: 0.0921
Batch 18900/76266, Loss: 0.1671
Batch 19000/76266, Loss: 0.2391
Batch 19100/76266, Loss: 0.2810
Batch 19200/76266, Loss: 0.1976
Batch 19300/76266, Loss: 0.3150
Batch 19400/76266, Loss: 0.1588
Batch 19500/76266, Loss: 0.3187
Batch 19600/76266, Loss: 0.3104
Batch 19700/76266, Loss: 0.2022
Batch 19800/76266, Loss: 0.1752
Batch 19900/76266, Loss: 0.1337
Batch 20000/76266, Loss: 0.1492
Batch 20100/76266, Loss: 0.2456
Batch 20200/76266, Loss: 0.2222

Batch 20200/76266, Loss: 0.2020
Batch 20300/76266, Loss: 0.2066
Batch 20400/76266, Loss: 0.1270
Batch 20500/76266, Loss: 0.1792
Batch 20600/76266, Loss: 0.2529
Batch 20700/76266, Loss: 0.2289
Batch 20800/76266, Loss: 0.2736
Batch 20900/76266, Loss: 0.1669
Batch 21000/76266, Loss: 0.3918
Batch 21100/76266, Loss: 0.2041
Batch 21200/76266, Loss: 0.1903
Batch 21300/76266, Loss: 0.1397
Batch 21400/76266, Loss: 0.2438
Batch 21500/76266, Loss: 0.2307
Batch 21600/76266, Loss: 0.1549
Batch 21700/76266, Loss: 0.2869
Batch 21800/76266, Loss: 0.1730
Batch 21900/76266, Loss: 0.2314
Batch 22000/76266, Loss: 0.1369
Batch 22100/76266, Loss: 0.1676
Batch 22200/76266, Loss: 0.2102
Batch 22300/76266, Loss: 0.2055
Batch 22400/76266, Loss: 0.2151
Batch 22500/76266, Loss: 0.1745
Batch 22600/76266, Loss: 0.1930
Batch 22700/76266, Loss: 0.1723
Batch 22800/76266, Loss: 0.3254
Batch 22900/76266, Loss: 0.1989
Batch 23000/76266, Loss: 0.2045
Batch 23100/76266, Loss: 0.1704
Batch 23200/76266, Loss: 0.1806
Batch 23300/76266, Loss: 0.1930
Batch 23400/76266, Loss: 0.1806
Batch 23500/76266, Loss: 0.2298
Batch 23600/76266, Loss: 0.1739
Batch 23700/76266, Loss: 0.2080
Batch 23800/76266, Loss: 0.2170
Batch 23900/76266, Loss: 0.2193
Batch 24000/76266, Loss: 0.1349
Batch 24100/76266, Loss: 0.2066
Batch 24200/76266, Loss: 0.2540
Batch 24300/76266, Loss: 0.1704
Batch 24400/76266, Loss: 0.2967
Batch 24500/76266, Loss: 0.2006
Batch 24600/76266, Loss: 0.1934
Batch 24700/76266, Loss: 0.2371
Batch 24800/76266, Loss: 0.1396
Batch 24900/76266, Loss: 0.2543
Batch 25000/76266, Loss: 0.1813
Batch 25100/76266, Loss: 0.2067
Batch 25200/76266, Loss: 0.1732
Batch 25300/76266, Loss: 0.1568
Batch 25400/76266, Loss: 0.1723
Batch 25500/76266, Loss: 0.1554
Batch 25600/76266, Loss: 0.2196
Batch 25700/76266, Loss: 0.2096
Batch 25800/76266, Loss: 0.1643

Batch 25900/76266, Loss: 0.2193
Batch 26000/76266, Loss: 0.2765
Batch 26100/76266, Loss: 0.2162
Batch 26200/76266, Loss: 0.0834
Batch 26300/76266, Loss: 0.1960
Batch 26400/76266, Loss: 0.1398
Batch 26500/76266, Loss: 0.1106
Batch 26600/76266, Loss: 0.1789
Batch 26700/76266, Loss: 0.2744
Batch 26800/76266, Loss: 0.2532
Batch 26900/76266, Loss: 0.2172
Batch 27000/76266, Loss: 0.2036
Batch 27100/76266, Loss: 0.1895
Batch 27200/76266, Loss: 0.2987
Batch 27300/76266, Loss: 0.2571
Batch 27400/76266, Loss: 0.1842
Batch 27500/76266, Loss: 0.2692
Batch 27600/76266, Loss: 0.1749
Batch 27700/76266, Loss: 0.2327
Batch 27800/76266, Loss: 0.2725
Batch 27900/76266, Loss: 0.2709
Batch 28000/76266, Loss: 0.2527
Batch 28100/76266, Loss: 0.1481
Batch 28200/76266, Loss: 0.1578
Batch 28300/76266, Loss: 0.3645
Batch 28400/76266, Loss: 0.2198
Batch 28500/76266, Loss: 0.1824
Batch 28600/76266, Loss: 0.1929
Batch 28700/76266, Loss: 0.1950
Batch 28800/76266, Loss: 0.2070
Batch 28900/76266, Loss: 0.2077
Batch 29000/76266, Loss: 0.2643
Batch 29100/76266, Loss: 0.2158
Batch 29200/76266, Loss: 0.2541
Batch 29300/76266, Loss: 0.2106
Batch 29400/76266, Loss: 0.1833
Batch 29500/76266, Loss: 0.2310
Batch 29600/76266, Loss: 0.1643
Batch 29700/76266, Loss: 0.1536
Batch 29800/76266, Loss: 0.2345
Batch 29900/76266, Loss: 0.1045
Batch 30000/76266, Loss: 0.2234
Batch 30100/76266, Loss: 0.1911
Batch 30200/76266, Loss: 0.2631
Batch 30300/76266, Loss: 0.1572
Batch 30400/76266, Loss: 0.2111
Batch 30500/76266, Loss: 0.2817
Batch 30600/76266, Loss: 0.1207
Batch 30700/76266, Loss: 0.2325
Batch 30800/76266, Loss: 0.2379
Batch 30900/76266, Loss: 0.2166
Batch 31000/76266, Loss: 0.1365
Batch 31100/76266, Loss: 0.1467
Batch 31200/76266, Loss: 0.1419
Batch 31300/76266, Loss: 0.1371
Batch 31400/76266, Loss: 0.1268
Batch 31500/76266, Loss: 0.1122

Batch 31500/76266, Loss: 0.1723
Batch 31600/76266, Loss: 0.3128
Batch 31700/76266, Loss: 0.2576
Batch 31800/76266, Loss: 0.1639
Batch 31900/76266, Loss: 0.1621
Batch 32000/76266, Loss: 0.1860
Batch 32100/76266, Loss: 0.1957
Batch 32200/76266, Loss: 0.1861
Batch 32300/76266, Loss: 0.2048
Batch 32400/76266, Loss: 0.1566
Batch 32500/76266, Loss: 0.2467
Batch 32600/76266, Loss: 0.1027
Batch 32700/76266, Loss: 0.1094
Batch 32800/76266, Loss: 0.2385
Batch 32900/76266, Loss: 0.2050
Batch 33000/76266, Loss: 0.3051
Batch 33100/76266, Loss: 0.1325
Batch 33200/76266, Loss: 0.1830
Batch 33300/76266, Loss: 0.1870
Batch 33400/76266, Loss: 0.1133
Batch 33500/76266, Loss: 0.2089
Batch 33600/76266, Loss: 0.2441
Batch 33700/76266, Loss: 0.2200
Batch 33800/76266, Loss: 0.1319
Batch 33900/76266, Loss: 0.1659
Batch 34000/76266, Loss: 0.1422
Batch 34100/76266, Loss: 0.1668
Batch 34200/76266, Loss: 0.1015
Batch 34300/76266, Loss: 0.1389
Batch 34400/76266, Loss: 0.2347
Batch 34500/76266, Loss: 0.2282
Batch 34600/76266, Loss: 0.2658
Batch 34700/76266, Loss: 0.1560
Batch 34800/76266, Loss: 0.1281
Batch 34900/76266, Loss: 0.3291
Batch 35000/76266, Loss: 0.1821
Batch 35100/76266, Loss: 0.2044
Batch 35200/76266, Loss: 0.1373
Batch 35300/76266, Loss: 0.1702
Batch 35400/76266, Loss: 0.2323
Batch 35500/76266, Loss: 0.1478
Batch 35600/76266, Loss: 0.1552
Batch 35700/76266, Loss: 0.1755
Batch 35800/76266, Loss: 0.1759
Batch 35900/76266, Loss: 0.2119
Batch 36000/76266, Loss: 0.2380
Batch 36100/76266, Loss: 0.1594
Batch 36200/76266, Loss: 0.1564
Batch 36300/76266, Loss: 0.1533
Batch 36400/76266, Loss: 0.1870
Batch 36500/76266, Loss: 0.1580
Batch 36600/76266, Loss: 0.1864
Batch 36700/76266, Loss: 0.1268
Batch 36800/76266, Loss: 0.2029
Batch 36900/76266, Loss: 0.1901
Batch 37000/76266, Loss: 0.2068
Batch 37100/76266, Loss: 0.3116

Batch 37200/76266, Loss: 0.1610
Batch 37300/76266, Loss: 0.2646
Batch 37400/76266, Loss: 0.2343
Batch 37500/76266, Loss: 0.2166
Batch 37600/76266, Loss: 0.2145
Batch 37700/76266, Loss: 0.1937
Batch 37800/76266, Loss: 0.2107
Batch 37900/76266, Loss: 0.3091
Batch 38000/76266, Loss: 0.1821
Batch 38100/76266, Loss: 0.0974
Batch 38200/76266, Loss: 0.2048
Batch 38300/76266, Loss: 0.1544
Batch 38400/76266, Loss: 0.1765
Batch 38500/76266, Loss: 0.2508
Batch 38600/76266, Loss: 0.2036
Batch 38700/76266, Loss: 0.1695
Batch 38800/76266, Loss: 0.1414
Batch 38900/76266, Loss: 0.2138
Batch 39000/76266, Loss: 0.2773
Batch 39100/76266, Loss: 0.1572
Batch 39200/76266, Loss: 0.2302
Batch 39300/76266, Loss: 0.1468
Batch 39400/76266, Loss: 0.2423
Batch 39500/76266, Loss: 0.2049
Batch 39600/76266, Loss: 0.2150
Batch 39700/76266, Loss: 0.2176
Batch 39800/76266, Loss: 0.2172
Batch 39900/76266, Loss: 0.1160
Batch 40000/76266, Loss: 0.2247
Batch 40100/76266, Loss: 0.1858
Batch 40200/76266, Loss: 0.1654
Batch 40300/76266, Loss: 0.1742
Batch 40400/76266, Loss: 0.2464
Batch 40500/76266, Loss: 0.2041
Batch 40600/76266, Loss: 0.1845
Batch 40700/76266, Loss: 0.2099
Batch 40800/76266, Loss: 0.2025
Batch 40900/76266, Loss: 0.2317
Batch 41000/76266, Loss: 0.1976
Batch 41100/76266, Loss: 0.1893
Batch 41200/76266, Loss: 0.1837
Batch 41300/76266, Loss: 0.1883
Batch 41400/76266, Loss: 0.2328
Batch 41500/76266, Loss: 0.1244
Batch 41600/76266, Loss: 0.2116
Batch 41700/76266, Loss: 0.1041
Batch 41800/76266, Loss: 0.1994
Batch 41900/76266, Loss: 0.1975
Batch 42000/76266, Loss: 0.1891
Batch 42100/76266, Loss: 0.1603
Batch 42200/76266, Loss: 0.2482
Batch 42300/76266, Loss: 0.1729
Batch 42400/76266, Loss: 0.1563
Batch 42500/76266, Loss: 0.2529
Batch 42600/76266, Loss: 0.2582
Batch 42700/76266, Loss: 0.2247
Batch 42800/76266, Loss: 0.2879

Batch 42900/76266, Loss: 0.1841
Batch 43000/76266, Loss: 0.2111
Batch 43100/76266, Loss: 0.1793
Batch 43200/76266, Loss: 0.1094
Batch 43300/76266, Loss: 0.1470
Batch 43400/76266, Loss: 0.2450
Batch 43500/76266, Loss: 0.1595
Batch 43600/76266, Loss: 0.1510
Batch 43700/76266, Loss: 0.1521
Batch 43800/76266, Loss: 0.1497
Batch 43900/76266, Loss: 0.2627
Batch 44000/76266, Loss: 0.2826
Batch 44100/76266, Loss: 0.1525
Batch 44200/76266, Loss: 0.2206
Batch 44300/76266, Loss: 0.4608
Batch 44400/76266, Loss: 0.1515
Batch 44500/76266, Loss: 0.2191
Batch 44600/76266, Loss: 0.1493
Batch 44700/76266, Loss: 0.3278
Batch 44800/76266, Loss: 0.1590
Batch 44900/76266, Loss: 0.1605
Batch 45000/76266, Loss: 0.1858
Batch 45100/76266, Loss: 0.1789
Batch 45200/76266, Loss: 0.1410
Batch 45300/76266, Loss: 0.2489
Batch 45400/76266, Loss: 0.1788
Batch 45500/76266, Loss: 0.1830
Batch 45600/76266, Loss: 0.2198
Batch 45700/76266, Loss: 0.1033
Batch 45800/76266, Loss: 0.2400
Batch 45900/76266, Loss: 0.1904
Batch 46000/76266, Loss: 0.0719
Batch 46100/76266, Loss: 0.1952
Batch 46200/76266, Loss: 0.1501
Batch 46300/76266, Loss: 0.2341
Batch 46400/76266, Loss: 0.1325
Batch 46500/76266, Loss: 0.1399
Batch 46600/76266, Loss: 0.1825
Batch 46700/76266, Loss: 0.1323
Batch 46800/76266, Loss: 0.1054
Batch 46900/76266, Loss: 0.1450
Batch 47000/76266, Loss: 0.2339
Batch 47100/76266, Loss: 0.1293
Batch 47200/76266, Loss: 0.2351
Batch 47300/76266, Loss: 0.1948
Batch 47400/76266, Loss: 0.2359
Batch 47500/76266, Loss: 0.1942
Batch 47600/76266, Loss: 0.1251
Batch 47700/76266, Loss: 0.1592
Batch 47800/76266, Loss: 0.1504
Batch 47900/76266, Loss: 0.1820
Batch 48000/76266, Loss: 0.1788
Batch 48100/76266, Loss: 0.2676
Batch 48200/76266, Loss: 0.1717
Batch 48300/76266, Loss: 0.1743
Batch 48400/76266, Loss: 0.2337

Batch 48500/76266, Loss: 0.2691
Batch 48600/76266, Loss: 0.2195
Batch 48700/76266, Loss: 0.1962
Batch 48800/76266, Loss: 0.2601
Batch 48900/76266, Loss: 0.1479
Batch 49000/76266, Loss: 0.1300
Batch 49100/76266, Loss: 0.2421
Batch 49200/76266, Loss: 0.1610
Batch 49300/76266, Loss: 0.1644
Batch 49400/76266, Loss: 0.1643
Batch 49500/76266, Loss: 0.2049
Batch 49600/76266, Loss: 0.1892
Batch 49700/76266, Loss: 0.2110
Batch 49800/76266, Loss: 0.2457
Batch 49900/76266, Loss: 0.2552
Batch 50000/76266, Loss: 0.2504
Batch 50100/76266, Loss: 0.2234
Batch 50200/76266, Loss: 0.1702
Batch 50300/76266, Loss: 0.2774
Batch 50400/76266, Loss: 0.2562
Batch 50500/76266, Loss: 0.1243
Batch 50600/76266, Loss: 0.1512
Batch 50700/76266, Loss: 0.0935
Batch 50800/76266, Loss: 0.1785
Batch 50900/76266, Loss: 0.1612
Batch 51000/76266, Loss: 0.3212
Batch 51100/76266, Loss: 0.2323
Batch 51200/76266, Loss: 0.2011
Batch 51300/76266, Loss: 0.1508
Batch 51400/76266, Loss: 0.1881
Batch 51500/76266, Loss: 0.1988
Batch 51600/76266, Loss: 0.1585
Batch 51700/76266, Loss: 0.2006
Batch 51800/76266, Loss: 0.2060
Batch 51900/76266, Loss: 0.2123
Batch 52000/76266, Loss: 0.2968
Batch 52100/76266, Loss: 0.2070
Batch 52200/76266, Loss: 0.2128
Batch 52300/76266, Loss: 0.1823
Batch 52400/76266, Loss: 0.1607
Batch 52500/76266, Loss: 0.2812
Batch 52600/76266, Loss: 0.1427
Batch 52700/76266, Loss: 0.1735
Batch 52800/76266, Loss: 0.3883
Batch 52900/76266, Loss: 0.1800
Batch 53000/76266, Loss: 0.2503
Batch 53100/76266, Loss: 0.1523
Batch 53200/76266, Loss: 0.2232
Batch 53300/76266, Loss: 0.3323
Batch 53400/76266, Loss: 0.1775
Batch 53500/76266, Loss: 0.2074
Batch 53600/76266, Loss: 0.1658
Batch 53700/76266, Loss: 0.1225
Batch 53800/76266, Loss: 0.2228
Batch 53900/76266, Loss: 0.2361
Batch 54000/76266, Loss: 0.1485
Batch 54100/76266, Loss: 0.0955

Batch 54200/76266, Loss: 0.2784
Batch 54300/76266, Loss: 0.1540
Batch 54400/76266, Loss: 0.1736
Batch 54500/76266, Loss: 0.1892
Batch 54600/76266, Loss: 0.1811
Batch 54700/76266, Loss: 0.1575
Batch 54800/76266, Loss: 0.1530
Batch 54900/76266, Loss: 0.1900
Batch 55000/76266, Loss: 0.1584
Batch 55100/76266, Loss: 0.1232
Batch 55200/76266, Loss: 0.2686
Batch 55300/76266, Loss: 0.1164
Batch 55400/76266, Loss: 0.2373
Batch 55500/76266, Loss: 0.1730
Batch 55600/76266, Loss: 0.2013
Batch 55700/76266, Loss: 0.2289
Batch 55800/76266, Loss: 0.2042
Batch 55900/76266, Loss: 0.1925
Batch 56000/76266, Loss: 0.1446
Batch 56100/76266, Loss: 0.2316
Batch 56200/76266, Loss: 0.1721
Batch 56300/76266, Loss: 0.1293
Batch 56400/76266, Loss: 0.1417
Batch 56500/76266, Loss: 0.1720
Batch 56600/76266, Loss: 0.1427
Batch 56700/76266, Loss: 0.1863
Batch 56800/76266, Loss: 0.2259
Batch 56900/76266, Loss: 0.1153
Batch 57000/76266, Loss: 0.2235
Batch 57100/76266, Loss: 0.1567
Batch 57200/76266, Loss: 0.2002
Batch 57300/76266, Loss: 0.2007
Batch 57400/76266, Loss: 0.1970
Batch 57500/76266, Loss: 0.3015
Batch 57600/76266, Loss: 0.1976
Batch 57700/76266, Loss: 0.1825
Batch 57800/76266, Loss: 0.1602
Batch 57900/76266, Loss: 0.2068
Batch 58000/76266, Loss: 0.3111
Batch 58100/76266, Loss: 0.1880
Batch 58200/76266, Loss: 0.1333
Batch 58300/76266, Loss: 0.1850
Batch 58400/76266, Loss: 0.2252
Batch 58500/76266, Loss: 0.2413
Batch 58600/76266, Loss: 0.1239
Batch 58700/76266, Loss: 0.2548
Batch 58800/76266, Loss: 0.2110
Batch 58900/76266, Loss: 0.1625
Batch 59000/76266, Loss: 0.3206
Batch 59100/76266, Loss: 0.2286
Batch 59200/76266, Loss: 0.2365
Batch 59300/76266, Loss: 0.2358
Batch 59400/76266, Loss: 0.2226
Batch 59500/76266, Loss: 0.2129
Batch 59600/76266, Loss: 0.3003
Batch 59700/76266, Loss: 0.1623

Batch 59800/76266, Loss: 0.1712
Batch 59900/76266, Loss: 0.2608
Batch 60000/76266, Loss: 0.2422
Batch 60100/76266, Loss: 0.1610
Batch 60200/76266, Loss: 0.2266
Batch 60300/76266, Loss: 0.3118
Batch 60400/76266, Loss: 0.1085
Batch 60500/76266, Loss: 0.2288
Batch 60600/76266, Loss: 0.1520
Batch 60700/76266, Loss: 0.1582
Batch 60800/76266, Loss: 0.2867
Batch 60900/76266, Loss: 0.1628
Batch 61000/76266, Loss: 0.1794
Batch 61100/76266, Loss: 0.1564
Batch 61200/76266, Loss: 0.1875
Batch 61300/76266, Loss: 0.2002
Batch 61400/76266, Loss: 0.2238
Batch 61500/76266, Loss: 0.1295
Batch 61600/76266, Loss: 0.1636
Batch 61700/76266, Loss: 0.2851
Batch 61800/76266, Loss: 0.3298
Batch 61900/76266, Loss: 0.3094
Batch 62000/76266, Loss: 0.2696
Batch 62100/76266, Loss: 0.1581
Batch 62200/76266, Loss: 0.1908
Batch 62300/76266, Loss: 0.2102
Batch 62400/76266, Loss: 0.2078
Batch 62500/76266, Loss: 0.2751
Batch 62600/76266, Loss: 0.2638
Batch 62700/76266, Loss: 0.1223
Batch 62800/76266, Loss: 0.1855
Batch 62900/76266, Loss: 0.2348
Batch 63000/76266, Loss: 0.1229
Batch 63100/76266, Loss: 0.2206
Batch 63200/76266, Loss: 0.1171
Batch 63300/76266, Loss: 0.2048
Batch 63400/76266, Loss: 0.2628
Batch 63500/76266, Loss: 0.2431
Batch 63600/76266, Loss: 0.1578
Batch 63700/76266, Loss: 0.1650
Batch 63800/76266, Loss: 0.2641
Batch 63900/76266, Loss: 0.1545
Batch 64000/76266, Loss: 0.2187
Batch 64100/76266, Loss: 0.3032
Batch 64200/76266, Loss: 0.1935
Batch 64300/76266, Loss: 0.1333
Batch 64400/76266, Loss: 0.2425
Batch 64500/76266, Loss: 0.2560
Batch 64600/76266, Loss: 0.1558
Batch 64700/76266, Loss: 0.2038
Batch 64800/76266, Loss: 0.1204
Batch 64900/76266, Loss: 0.0650
Batch 65000/76266, Loss: 0.1162
Batch 65100/76266, Loss: 0.3359
Batch 65200/76266, Loss: 0.3680
Batch 65300/76266, Loss: 0.2217
Batch 65400/76266, Loss: 0.1681

Batch 65500/76266, Loss: 0.2540
Batch 65600/76266, Loss: 0.2484
Batch 65700/76266, Loss: 0.2583
Batch 65800/76266, Loss: 0.1621
Batch 65900/76266, Loss: 0.1115
Batch 66000/76266, Loss: 0.2790
Batch 66100/76266, Loss: 0.1974
Batch 66200/76266, Loss: 0.3531
Batch 66300/76266, Loss: 0.1371
Batch 66400/76266, Loss: 0.3072
Batch 66500/76266, Loss: 0.1466
Batch 66600/76266, Loss: 0.1399
Batch 66700/76266, Loss: 0.2378
Batch 66800/76266, Loss: 0.2150
Batch 66900/76266, Loss: 0.2196
Batch 67000/76266, Loss: 0.2396
Batch 67100/76266, Loss: 0.1815
Batch 67200/76266, Loss: 0.1190
Batch 67300/76266, Loss: 0.1916
Batch 67400/76266, Loss: 0.2017
Batch 67500/76266, Loss: 0.1218
Batch 67600/76266, Loss: 0.1471
Batch 67700/76266, Loss: 0.2201
Batch 67800/76266, Loss: 0.2741
Batch 67900/76266, Loss: 0.1468
Batch 68000/76266, Loss: 0.2631
Batch 68100/76266, Loss: 0.2291
Batch 68200/76266, Loss: 0.2040
Batch 68300/76266, Loss: 0.1624
Batch 68400/76266, Loss: 0.2113
Batch 68500/76266, Loss: 0.2675
Batch 68600/76266, Loss: 0.1420
Batch 68700/76266, Loss: 0.1066
Batch 68800/76266, Loss: 0.2612
Batch 68900/76266, Loss: 0.2464
Batch 69000/76266, Loss: 0.1354
Batch 69100/76266, Loss: 0.1768
Batch 69200/76266, Loss: 0.3263
Batch 69300/76266, Loss: 0.1345
Batch 69400/76266, Loss: 0.1291
Batch 69500/76266, Loss: 0.1810
Batch 69600/76266, Loss: 0.1641
Batch 69700/76266, Loss: 0.2637
Batch 69800/76266, Loss: 0.2867
Batch 69900/76266, Loss: 0.1829
Batch 70000/76266, Loss: 0.2391
Batch 70100/76266, Loss: 0.1725
Batch 70200/76266, Loss: 0.1446
Batch 70300/76266, Loss: 0.2972
Batch 70400/76266, Loss: 0.1746
Batch 70500/76266, Loss: 0.1661
Batch 70600/76266, Loss: 0.2459
Batch 70700/76266, Loss: 0.2377
Batch 70800/76266, Loss: 0.1397
Batch 70900/76266, Loss: 0.2857
Batch 71000/76266, Loss: 0.2010

Batch 71100/76266, Loss: 0.2043
Batch 71200/76266, Loss: 0.1699
Batch 71300/76266, Loss: 0.2112
Batch 71400/76266, Loss: 0.2498
Batch 71500/76266, Loss: 0.2223
Batch 71600/76266, Loss: 0.2399
Batch 71700/76266, Loss: 0.1318
Batch 71800/76266, Loss: 0.2654
Batch 71900/76266, Loss: 0.2251
Batch 72000/76266, Loss: 0.1949
Batch 72100/76266, Loss: 0.1900
Batch 72200/76266, Loss: 0.2093
Batch 72300/76266, Loss: 0.2615
Batch 72400/76266, Loss: 0.2174
Batch 72500/76266, Loss: 0.2776
Batch 72600/76266, Loss: 0.1061
Batch 72700/76266, Loss: 0.1561
Batch 72800/76266, Loss: 0.1214
Batch 72900/76266, Loss: 0.1538
Batch 73000/76266, Loss: 0.1539
Batch 73100/76266, Loss: 0.1732
Batch 73200/76266, Loss: 0.3395
Batch 73300/76266, Loss: 0.1320
Batch 73400/76266, Loss: 0.2431
Batch 73500/76266, Loss: 0.1835
Batch 73600/76266, Loss: 0.1275
Batch 73700/76266, Loss: 0.1961
Batch 73800/76266, Loss: 0.1406
Batch 73900/76266, Loss: 0.1540
Batch 74000/76266, Loss: 0.2782
Batch 74100/76266, Loss: 0.1416
Batch 74200/76266, Loss: 0.2105
Batch 74300/76266, Loss: 0.1659
Batch 74400/76266, Loss: 0.2720
Batch 74500/76266, Loss: 0.1487
Batch 74600/76266, Loss: 0.1369
Batch 74700/76266, Loss: 0.2797
Batch 74800/76266, Loss: 0.1629
Batch 74900/76266, Loss: 0.2112
Batch 75000/76266, Loss: 0.1013
Batch 75100/76266, Loss: 0.2105
Batch 75200/76266, Loss: 0.2648
Batch 75300/76266, Loss: 0.1598
Batch 75400/76266, Loss: 0.1933
Batch 75500/76266, Loss: 0.2037
Batch 75600/76266, Loss: 0.0774
Batch 75700/76266, Loss: 0.1475
Batch 75800/76266, Loss: 0.2377
Batch 75900/76266, Loss: 0.1595
Batch 76000/76266, Loss: 0.1273
Batch 76100/76266, Loss: 0.1977
Batch 76200/76266, Loss: 0.1651

Epoch 2 completed in 11581.31s. Average Training Loss: 0.2025

--- Epoch 3/3 ---

Batch 100/76266, Loss: 0.1581
Batch 200/76266, Loss: 0.1791
Batch 300/76266, Loss: 0.1786

Batch 400/76266, Loss: 0.2439
Batch 500/76266, Loss: 0.1951
Batch 600/76266, Loss: 0.2263
Batch 700/76266, Loss: 0.1488
Batch 800/76266, Loss: 0.0846
Batch 900/76266, Loss: 0.1654
Batch 1000/76266, Loss: 0.1629
Batch 1100/76266, Loss: 0.2740
Batch 1200/76266, Loss: 0.1396
Batch 1300/76266, Loss: 0.2975
Batch 1400/76266, Loss: 0.2110
Batch 1500/76266, Loss: 0.0947
Batch 1600/76266, Loss: 0.0677
Batch 1700/76266, Loss: 0.2375
Batch 1800/76266, Loss: 0.2209
Batch 1900/76266, Loss: 0.2338
Batch 2000/76266, Loss: 0.0951
Batch 2100/76266, Loss: 0.2041
Batch 2200/76266, Loss: 0.2131
Batch 2300/76266, Loss: 0.2192
Batch 2400/76266, Loss: 0.1516
Batch 2500/76266, Loss: 0.2237
Batch 2600/76266, Loss: 0.1795
Batch 2700/76266, Loss: 0.2098
Batch 2800/76266, Loss: 0.1985
Batch 2900/76266, Loss: 0.1628
Batch 3000/76266, Loss: 0.1422
Batch 3100/76266, Loss: 0.1531
Batch 3200/76266, Loss: 0.1347
Batch 3300/76266, Loss: 0.2064
Batch 3400/76266, Loss: 0.1842
Batch 3500/76266, Loss: 0.1506
Batch 3600/76266, Loss: 0.2122
Batch 3700/76266, Loss: 0.1374
Batch 3800/76266, Loss: 0.1611
Batch 3900/76266, Loss: 0.2446
Batch 4000/76266, Loss: 0.1855
Batch 4100/76266, Loss: 0.1359
Batch 4200/76266, Loss: 0.1747
Batch 4300/76266, Loss: 0.1375
Batch 4400/76266, Loss: 0.2465
Batch 4500/76266, Loss: 0.1958
Batch 4600/76266, Loss: 0.1785
Batch 4700/76266, Loss: 0.3493
Batch 4800/76266, Loss: 0.1655
Batch 4900/76266, Loss: 0.1167
Batch 5000/76266, Loss: 0.2245
Batch 5100/76266, Loss: 0.1526
Batch 5200/76266, Loss: 0.1718
Batch 5300/76266, Loss: 0.2206
Batch 5400/76266, Loss: 0.2017
Batch 5500/76266, Loss: 0.1409
Batch 5600/76266, Loss: 0.1853
Batch 5700/76266, Loss: 0.1557
Batch 5800/76266, Loss: 0.1173
Batch 5900/76266, Loss: 0.0767

Batch 6000/76266, Loss: 0.2415
Batch 6100/76266, Loss: 0.1893
Batch 6200/76266, Loss: 0.1193
Batch 6300/76266, Loss: 0.1577
Batch 6400/76266, Loss: 0.1626
Batch 6500/76266, Loss: 0.0796
Batch 6600/76266, Loss: 0.1856
Batch 6700/76266, Loss: 0.2947
Batch 6800/76266, Loss: 0.1652
Batch 6900/76266, Loss: 0.1819
Batch 7000/76266, Loss: 0.1986
Batch 7100/76266, Loss: 0.1830
Batch 7200/76266, Loss: 0.0828
Batch 7300/76266, Loss: 0.2284
Batch 7400/76266, Loss: 0.2144
Batch 7500/76266, Loss: 0.2210
Batch 7600/76266, Loss: 0.2809
Batch 7700/76266, Loss: 0.1586
Batch 7800/76266, Loss: 0.1357
Batch 7900/76266, Loss: 0.1394
Batch 8000/76266, Loss: 0.1984
Batch 8100/76266, Loss: 0.1724
Batch 8200/76266, Loss: 0.1500
Batch 8300/76266, Loss: 0.1405
Batch 8400/76266, Loss: 0.1387
Batch 8500/76266, Loss: 0.0649
Batch 8600/76266, Loss: 0.2856
Batch 8700/76266, Loss: 0.1854
Batch 8800/76266, Loss: 0.2356
Batch 8900/76266, Loss: 0.1878
Batch 9000/76266, Loss: 0.1261
Batch 9100/76266, Loss: 0.1773
Batch 9200/76266, Loss: 0.2432
Batch 9300/76266, Loss: 0.0786
Batch 9400/76266, Loss: 0.2285
Batch 9500/76266, Loss: 0.1129
Batch 9600/76266, Loss: 0.2016
Batch 9700/76266, Loss: 0.2026
Batch 9800/76266, Loss: 0.0923
Batch 9900/76266, Loss: 0.3019
Batch 10000/76266, Loss: 0.2178
Batch 10100/76266, Loss: 0.1692
Batch 10200/76266, Loss: 0.1216
Batch 10300/76266, Loss: 0.3508
Batch 10400/76266, Loss: 0.1884
Batch 10500/76266, Loss: 0.1272
Batch 10600/76266, Loss: 0.1699
Batch 10700/76266, Loss: 0.2636
Batch 10800/76266, Loss: 0.1875
Batch 10900/76266, Loss: 0.1464
Batch 11000/76266, Loss: 0.1026
Batch 11100/76266, Loss: 0.1775
Batch 11200/76266, Loss: 0.1364
Batch 11300/76266, Loss: 0.2127
Batch 11400/76266, Loss: 0.1443
Batch 11500/76266, Loss: 0.1345
Batch 11600/76266, Loss: 0.2193

Batch 11700/76266, Loss: 0.1877
Batch 11800/76266, Loss: 0.2652
Batch 11900/76266, Loss: 0.1645
Batch 12000/76266, Loss: 0.1680
Batch 12100/76266, Loss: 0.1260
Batch 12200/76266, Loss: 0.1203
Batch 12300/76266, Loss: 0.1548
Batch 12400/76266, Loss: 0.1169
Batch 12500/76266, Loss: 0.1734
Batch 12600/76266, Loss: 0.2050
Batch 12700/76266, Loss: 0.1184
Batch 12800/76266, Loss: 0.1856
Batch 12900/76266, Loss: 0.2727
Batch 13000/76266, Loss: 0.1386
Batch 13100/76266, Loss: 0.1717
Batch 13200/76266, Loss: 0.1484
Batch 13300/76266, Loss: 0.1874
Batch 13400/76266, Loss: 0.1470
Batch 13500/76266, Loss: 0.1175
Batch 13600/76266, Loss: 0.1090
Batch 13700/76266, Loss: 0.2962
Batch 13800/76266, Loss: 0.1302
Batch 13900/76266, Loss: 0.2048
Batch 14000/76266, Loss: 0.1433
Batch 14100/76266, Loss: 0.1732
Batch 14200/76266, Loss: 0.1446
Batch 14300/76266, Loss: 0.1838
Batch 14400/76266, Loss: 0.1290
Batch 14500/76266, Loss: 0.1095
Batch 14600/76266, Loss: 0.2538
Batch 14700/76266, Loss: 0.1637
Batch 14800/76266, Loss: 0.1752
Batch 14900/76266, Loss: 0.1958
Batch 15000/76266, Loss: 0.1487
Batch 15100/76266, Loss: 0.1466
Batch 15200/76266, Loss: 0.2118
Batch 15300/76266, Loss: 0.2186
Batch 15400/76266, Loss: 0.1903
Batch 15500/76266, Loss: 0.1643
Batch 15600/76266, Loss: 0.1612
Batch 15700/76266, Loss: 0.2397
Batch 15800/76266, Loss: 0.2024
Batch 15900/76266, Loss: 0.0955
Batch 16000/76266, Loss: 0.1974
Batch 16100/76266, Loss: 0.1941
Batch 16200/76266, Loss: 0.1691
Batch 16300/76266, Loss: 0.2059
Batch 16400/76266, Loss: 0.1133
Batch 16500/76266, Loss: 0.0716
Batch 16600/76266, Loss: 0.1284
Batch 16700/76266, Loss: 0.2831
Batch 16800/76266, Loss: 0.2005
Batch 16900/76266, Loss: 0.1505
Batch 17000/76266, Loss: 0.1664
Batch 17100/76266, Loss: 0.2478
Batch 17200/76266, Loss: 0.2913

Batch 17300/76266, Loss: 0.0807
Batch 17400/76266, Loss: 0.1836
Batch 17500/76266, Loss: 0.1720
Batch 17600/76266, Loss: 0.1578
Batch 17700/76266, Loss: 0.1883
Batch 17800/76266, Loss: 0.1679
Batch 17900/76266, Loss: 0.1404
Batch 18000/76266, Loss: 0.2073
Batch 18100/76266, Loss: 0.2329
Batch 18200/76266, Loss: 0.2477
Batch 18300/76266, Loss: 0.0795
Batch 18400/76266, Loss: 0.2354
Batch 18500/76266, Loss: 0.1566
Batch 18600/76266, Loss: 0.1700
Batch 18700/76266, Loss: 0.1390
Batch 18800/76266, Loss: 0.2924
Batch 18900/76266, Loss: 0.1884
Batch 19000/76266, Loss: 0.2771
Batch 19100/76266, Loss: 0.1450
Batch 19200/76266, Loss: 0.1902
Batch 19300/76266, Loss: 0.1708
Batch 19400/76266, Loss: 0.1145
Batch 19500/76266, Loss: 0.1395
Batch 19600/76266, Loss: 0.1215
Batch 19700/76266, Loss: 0.1681
Batch 19800/76266, Loss: 0.1331
Batch 19900/76266, Loss: 0.1430
Batch 20000/76266, Loss: 0.1714
Batch 20100/76266, Loss: 0.2711
Batch 20200/76266, Loss: 0.1643
Batch 20300/76266, Loss: 0.1828
Batch 20400/76266, Loss: 0.1796
Batch 20500/76266, Loss: 0.1485
Batch 20600/76266, Loss: 0.2072
Batch 20700/76266, Loss: 0.1629
Batch 20800/76266, Loss: 0.1367
Batch 20900/76266, Loss: 0.1467
Batch 21000/76266, Loss: 0.1493
Batch 21100/76266, Loss: 0.1354
Batch 21200/76266, Loss: 0.2494
Batch 21300/76266, Loss: 0.2266
Batch 21400/76266, Loss: 0.1950
Batch 21500/76266, Loss: 0.2310
Batch 21600/76266, Loss: 0.1841
Batch 21700/76266, Loss: 0.3002
Batch 21800/76266, Loss: 0.1478
Batch 21900/76266, Loss: 0.1552
Batch 22000/76266, Loss: 0.1516
Batch 22100/76266, Loss: 0.2016
Batch 22200/76266, Loss: 0.1427
Batch 22300/76266, Loss: 0.2902
Batch 22400/76266, Loss: 0.0887
Batch 22500/76266, Loss: 0.2228
Batch 22600/76266, Loss: 0.1765
Batch 22700/76266, Loss: 0.1939
Batch 22800/76266, Loss: 0.1075
Batch 22900/76266, Loss: 0.2396

Batch 23000/76266, Loss: 0.1144
Batch 23100/76266, Loss: 0.2000
Batch 23200/76266, Loss: 0.1142
Batch 23300/76266, Loss: 0.1235
Batch 23400/76266, Loss: 0.2136
Batch 23500/76266, Loss: 0.1726
Batch 23600/76266, Loss: 0.1024
Batch 23700/76266, Loss: 0.1384
Batch 23800/76266, Loss: 0.1493
Batch 23900/76266, Loss: 0.1560
Batch 24000/76266, Loss: 0.1564
Batch 24100/76266, Loss: 0.1001
Batch 24200/76266, Loss: 0.1590
Batch 24300/76266, Loss: 0.2751
Batch 24400/76266, Loss: 0.2335
Batch 24500/76266, Loss: 0.1656
Batch 24600/76266, Loss: 0.1594
Batch 24700/76266, Loss: 0.0690
Batch 24800/76266, Loss: 0.1554
Batch 24900/76266, Loss: 0.1694
Batch 25000/76266, Loss: 0.1994
Batch 25100/76266, Loss: 0.1418
Batch 25200/76266, Loss: 0.3240
Batch 25300/76266, Loss: 0.2336
Batch 25400/76266, Loss: 0.3899
Batch 25500/76266, Loss: 0.2423
Batch 25600/76266, Loss: 0.1184
Batch 25700/76266, Loss: 0.1858
Batch 25800/76266, Loss: 0.1394
Batch 25900/76266, Loss: 0.2004
Batch 26000/76266, Loss: 0.1143
Batch 26100/76266, Loss: 0.1479
Batch 26200/76266, Loss: 0.2519
Batch 26300/76266, Loss: 0.2261
Batch 26400/76266, Loss: 0.2557
Batch 26500/76266, Loss: 0.2641
Batch 26600/76266, Loss: 0.1665
Batch 26700/76266, Loss: 0.1873
Batch 26800/76266, Loss: 0.1179
Batch 26900/76266, Loss: 0.1460
Batch 27000/76266, Loss: 0.1597
Batch 27100/76266, Loss: 0.2074
Batch 27200/76266, Loss: 0.2431
Batch 27300/76266, Loss: 0.1395
Batch 27400/76266, Loss: 0.2365
Batch 27500/76266, Loss: 0.1801
Batch 27600/76266, Loss: 0.1911
Batch 27700/76266, Loss: 0.1964
Batch 27800/76266, Loss: 0.1699
Batch 27900/76266, Loss: 0.1303
Batch 28000/76266, Loss: 0.1639
Batch 28100/76266, Loss: 0.1943
Batch 28200/76266, Loss: 0.1683
Batch 28300/76266, Loss: 0.2270
Batch 28400/76266, Loss: 0.2202
Batch 28500/76266, Loss: 0.1605

Batch 28600/76266, Loss: 0.2770
Batch 28700/76266, Loss: 0.1436
Batch 28800/76266, Loss: 0.1650
Batch 28900/76266, Loss: 0.1715
Batch 29000/76266, Loss: 0.1743
Batch 29100/76266, Loss: 0.1267
Batch 29200/76266, Loss: 0.1041
Batch 29300/76266, Loss: 0.1363
Batch 29400/76266, Loss: 0.1655
Batch 29500/76266, Loss: 0.2527
Batch 29600/76266, Loss: 0.1861
Batch 29700/76266, Loss: 0.2889
Batch 29800/76266, Loss: 0.1403
Batch 29900/76266, Loss: 0.2383
Batch 30000/76266, Loss: 0.2316
Batch 30100/76266, Loss: 0.2950
Batch 30200/76266, Loss: 0.1600
Batch 30300/76266, Loss: 0.1600
Batch 30400/76266, Loss: 0.2163
Batch 30500/76266, Loss: 0.1205
Batch 30600/76266, Loss: 0.1163
Batch 30700/76266, Loss: 0.1848
Batch 30800/76266, Loss: 0.2466
Batch 30900/76266, Loss: 0.1692
Batch 31000/76266, Loss: 0.2178
Batch 31100/76266, Loss: 0.2126
Batch 31200/76266, Loss: 0.2211
Batch 31300/76266, Loss: 0.2566
Batch 31400/76266, Loss: 0.1487
Batch 31500/76266, Loss: 0.1353
Batch 31600/76266, Loss: 0.1236
Batch 31700/76266, Loss: 0.1355
Batch 31800/76266, Loss: 0.1952
Batch 31900/76266, Loss: 0.0917
Batch 32000/76266, Loss: 0.1447
Batch 32100/76266, Loss: 0.1361
Batch 32200/76266, Loss: 0.0964
Batch 32300/76266, Loss: 0.1499
Batch 32400/76266, Loss: 0.1890
Batch 32500/76266, Loss: 0.1284
Batch 32600/76266, Loss: 0.1750
Batch 32700/76266, Loss: 0.1470
Batch 32800/76266, Loss: 0.1303
Batch 32900/76266, Loss: 0.2102
Batch 33000/76266, Loss: 0.2011
Batch 33100/76266, Loss: 0.1740
Batch 33200/76266, Loss: 0.1211
Batch 33300/76266, Loss: 0.1981
Batch 33400/76266, Loss: 0.1180
Batch 33500/76266, Loss: 0.1861
Batch 33600/76266, Loss: 0.2186
Batch 33700/76266, Loss: 0.1421
Batch 33800/76266, Loss: 0.2474
Batch 33900/76266, Loss: 0.1757
Batch 34000/76266, Loss: 0.2308
Batch 34100/76266, Loss: 0.2029
Batch 34200/76266, Loss: 0.2543

Batch 34300/76266, Loss: 0.1993
Batch 34400/76266, Loss: 0.1954
Batch 34500/76266, Loss: 0.1425
Batch 34600/76266, Loss: 0.2173
Batch 34700/76266, Loss: 0.0877
Batch 34800/76266, Loss: 0.1930
Batch 34900/76266, Loss: 0.3169
Batch 35000/76266, Loss: 0.1263
Batch 35100/76266, Loss: 0.1919
Batch 35200/76266, Loss: 0.1659
Batch 35300/76266, Loss: 0.2136
Batch 35400/76266, Loss: 0.1990
Batch 35500/76266, Loss: 0.2053
Batch 35600/76266, Loss: 0.2166
Batch 35700/76266, Loss: 0.1207
Batch 35800/76266, Loss: 0.1650
Batch 35900/76266, Loss: 0.1164
Batch 36000/76266, Loss: 0.1483
Batch 36100/76266, Loss: 0.1688
Batch 36200/76266, Loss: 0.1213
Batch 36300/76266, Loss: 0.1765
Batch 36400/76266, Loss: 0.1696
Batch 36500/76266, Loss: 0.2185
Batch 36600/76266, Loss: 0.2054
Batch 36700/76266, Loss: 0.0982
Batch 36800/76266, Loss: 0.2187
Batch 36900/76266, Loss: 0.2417
Batch 37000/76266, Loss: 0.1518
Batch 37100/76266, Loss: 0.1491
Batch 37200/76266, Loss: 0.1025
Batch 37300/76266, Loss: 0.1788
Batch 37400/76266, Loss: 0.1377
Batch 37500/76266, Loss: 0.1143
Batch 37600/76266, Loss: 0.1330
Batch 37700/76266, Loss: 0.1795
Batch 37800/76266, Loss: 0.1962
Batch 37900/76266, Loss: 0.1230
Batch 38000/76266, Loss: 0.1981
Batch 38100/76266, Loss: 0.1498
Batch 38200/76266, Loss: 0.1346
Batch 38300/76266, Loss: 0.2046
Batch 38400/76266, Loss: 0.2033
Batch 38500/76266, Loss: 0.1635
Batch 38600/76266, Loss: 0.2038
Batch 38700/76266, Loss: 0.1693
Batch 38800/76266, Loss: 0.1369
Batch 38900/76266, Loss: 0.1648
Batch 39000/76266, Loss: 0.2025
Batch 39100/76266, Loss: 0.1439
Batch 39200/76266, Loss: 0.2570
Batch 39300/76266, Loss: 0.1755
Batch 39400/76266, Loss: 0.3515
Batch 39500/76266, Loss: 0.1906
Batch 39600/76266, Loss: 0.1984
Batch 39700/76266, Loss: 0.1755
Batch 39800/76266, Loss: 0.1732

Batch 39900/76266, Loss: 0.2419
Batch 40000/76266, Loss: 0.2069
Batch 40100/76266, Loss: 0.1721
Batch 40200/76266, Loss: 0.1288
Batch 40300/76266, Loss: 0.1867
Batch 40400/76266, Loss: 0.1269
Batch 40500/76266, Loss: 0.1695
Batch 40600/76266, Loss: 0.1934
Batch 40700/76266, Loss: 0.0797
Batch 40800/76266, Loss: 0.1942
Batch 40900/76266, Loss: 0.0908
Batch 41000/76266, Loss: 0.1634
Batch 41100/76266, Loss: 0.0987
Batch 41200/76266, Loss: 0.2477
Batch 41300/76266, Loss: 0.1732
Batch 41400/76266, Loss: 0.1650
Batch 41500/76266, Loss: 0.1687
Batch 41600/76266, Loss: 0.2566
Batch 41700/76266, Loss: 0.2025
Batch 41800/76266, Loss: 0.3054
Batch 41900/76266, Loss: 0.1240
Batch 42000/76266, Loss: 0.1721
Batch 42100/76266, Loss: 0.2391
Batch 42200/76266, Loss: 0.2507
Batch 42300/76266, Loss: 0.1906
Batch 42400/76266, Loss: 0.2789
Batch 42500/76266, Loss: 0.2084
Batch 42600/76266, Loss: 0.1830
Batch 42700/76266, Loss: 0.1617
Batch 42800/76266, Loss: 0.1753
Batch 42900/76266, Loss: 0.1376
Batch 43000/76266, Loss: 0.1579
Batch 43100/76266, Loss: 0.1763
Batch 43200/76266, Loss: 0.2176
Batch 43300/76266, Loss: 0.2086
Batch 43400/76266, Loss: 0.2219
Batch 43500/76266, Loss: 0.2248
Batch 43600/76266, Loss: 0.1598
Batch 43700/76266, Loss: 0.2190
Batch 43800/76266, Loss: 0.1564
Batch 43900/76266, Loss: 0.1107
Batch 44000/76266, Loss: 0.0891
Batch 44100/76266, Loss: 0.1491
Batch 44200/76266, Loss: 0.1798
Batch 44300/76266, Loss: 0.2464
Batch 44400/76266, Loss: 0.1294
Batch 44500/76266, Loss: 0.1797
Batch 44600/76266, Loss: 0.1878
Batch 44700/76266, Loss: 0.0936
Batch 44800/76266, Loss: 0.2262
Batch 44900/76266, Loss: 0.2106
Batch 45000/76266, Loss: 0.1208
Batch 45100/76266, Loss: 0.1266
Batch 45200/76266, Loss: 0.1098
Batch 45300/76266, Loss: 0.3071
Batch 45400/76266, Loss: 0.2596
Batch 45500/76266, Loss: 0.0988

Batch 45600/76266, Loss: 0.2109
Batch 45700/76266, Loss: 0.2610
Batch 45800/76266, Loss: 0.0938
Batch 45900/76266, Loss: 0.2271
Batch 46000/76266, Loss: 0.2525
Batch 46100/76266, Loss: 0.1830
Batch 46200/76266, Loss: 0.1974
Batch 46300/76266, Loss: 0.1135
Batch 46400/76266, Loss: 0.3177
Batch 46500/76266, Loss: 0.2655
Batch 46600/76266, Loss: 0.2048
Batch 46700/76266, Loss: 0.1050
Batch 46800/76266, Loss: 0.1049
Batch 46900/76266, Loss: 0.2550
Batch 47000/76266, Loss: 0.2629
Batch 47100/76266, Loss: 0.2220
Batch 47200/76266, Loss: 0.2096
Batch 47300/76266, Loss: 0.2435
Batch 47400/76266, Loss: 0.1290
Batch 47500/76266, Loss: 0.1859
Batch 47600/76266, Loss: 0.1672
Batch 47700/76266, Loss: 0.2010
Batch 47800/76266, Loss: 0.0908
Batch 47900/76266, Loss: 0.1307
Batch 48000/76266, Loss: 0.1192
Batch 48100/76266, Loss: 0.0547
Batch 48200/76266, Loss: 0.2450
Batch 48300/76266, Loss: 0.2296
Batch 48400/76266, Loss: 0.1845
Batch 48500/76266, Loss: 0.1757
Batch 48600/76266, Loss: 0.1624
Batch 48700/76266, Loss: 0.2234
Batch 48800/76266, Loss: 0.2576
Batch 48900/76266, Loss: 0.2340
Batch 49000/76266, Loss: 0.1976
Batch 49100/76266, Loss: 0.1866
Batch 49200/76266, Loss: 0.1524
Batch 49300/76266, Loss: 0.1592
Batch 49400/76266, Loss: 0.0386
Batch 49500/76266, Loss: 0.1918
Batch 49600/76266, Loss: 0.1464
Batch 49700/76266, Loss: 0.1852
Batch 49800/76266, Loss: 0.1855
Batch 49900/76266, Loss: 0.1708
Batch 50000/76266, Loss: 0.1091
Batch 50100/76266, Loss: 0.2107
Batch 50200/76266, Loss: 0.1630
Batch 50300/76266, Loss: 0.0998
Batch 50400/76266, Loss: 0.2263
Batch 50500/76266, Loss: 0.1195
Batch 50600/76266, Loss: 0.1642
Batch 50700/76266, Loss: 0.1264
Batch 50800/76266, Loss: 0.2163
Batch 50900/76266, Loss: 0.2478
Batch 51000/76266, Loss: 0.1749
Batch 51100/76266, Loss: 0.1487

Batch 51200/76266, Loss: 0.1491
Batch 51300/76266, Loss: 0.1690
Batch 51400/76266, Loss: 0.1742
Batch 51500/76266, Loss: 0.1755
Batch 51600/76266, Loss: 0.2299
Batch 51700/76266, Loss: 0.1567
Batch 51800/76266, Loss: 0.1942
Batch 51900/76266, Loss: 0.1198
Batch 52000/76266, Loss: 0.2700
Batch 52100/76266, Loss: 0.1869
Batch 52200/76266, Loss: 0.1985
Batch 52300/76266, Loss: 0.1785
Batch 52400/76266, Loss: 0.1551
Batch 52500/76266, Loss: 0.1803
Batch 52600/76266, Loss: 0.1407
Batch 52700/76266, Loss: 0.2934
Batch 52800/76266, Loss: 0.1294
Batch 52900/76266, Loss: 0.0826
Batch 53000/76266, Loss: 0.2353
Batch 53100/76266, Loss: 0.0595
Batch 53200/76266, Loss: 0.1442
Batch 53300/76266, Loss: 0.2424
Batch 53400/76266, Loss: 0.2281
Batch 53500/76266, Loss: 0.1577
Batch 53600/76266, Loss: 0.1463
Batch 53700/76266, Loss: 0.1866
Batch 53800/76266, Loss: 0.1295
Batch 53900/76266, Loss: 0.1661
Batch 54000/76266, Loss: 0.1532
Batch 54100/76266, Loss: 0.1524
Batch 54200/76266, Loss: 0.1358
Batch 54300/76266, Loss: 0.2067
Batch 54400/76266, Loss: 0.1986
Batch 54500/76266, Loss: 0.2594
Batch 54600/76266, Loss: 0.2039
Batch 54700/76266, Loss: 0.2112
Batch 54800/76266, Loss: 0.2120
Batch 54900/76266, Loss: 0.2314
Batch 55000/76266, Loss: 0.2318
Batch 55100/76266, Loss: 0.3429
Batch 55200/76266, Loss: 0.1569
Batch 55300/76266, Loss: 0.2572
Batch 55400/76266, Loss: 0.2693
Batch 55500/76266, Loss: 0.3405
Batch 55600/76266, Loss: 0.1989
Batch 55700/76266, Loss: 0.2576
Batch 55800/76266, Loss: 0.2137
Batch 55900/76266, Loss: 0.2578
Batch 56000/76266, Loss: 0.1531
Batch 56100/76266, Loss: 0.0867
Batch 56200/76266, Loss: 0.1454
Batch 56300/76266, Loss: 0.1949
Batch 56400/76266, Loss: 0.1895
Batch 56500/76266, Loss: 0.1686
Batch 56600/76266, Loss: 0.1690
Batch 56700/76266, Loss: 0.1353
Batch 56800/76266, Loss: 0.1110

Batch 56900/76266, Loss: 0.1419
Batch 57000/76266, Loss: 0.2889
Batch 57100/76266, Loss: 0.1163
Batch 57200/76266, Loss: 0.1881
Batch 57300/76266, Loss: 0.2382
Batch 57400/76266, Loss: 0.2920
Batch 57500/76266, Loss: 0.1641
Batch 57600/76266, Loss: 0.2417
Batch 57700/76266, Loss: 0.1422
Batch 57800/76266, Loss: 0.1403
Batch 57900/76266, Loss: 0.1785
Batch 58000/76266, Loss: 0.1920
Batch 58100/76266, Loss: 0.0901
Batch 58200/76266, Loss: 0.2029
Batch 58300/76266, Loss: 0.1257
Batch 58400/76266, Loss: 0.2484
Batch 58500/76266, Loss: 0.1791
Batch 58600/76266, Loss: 0.2057
Batch 58700/76266, Loss: 0.3234
Batch 58800/76266, Loss: 0.1400
Batch 58900/76266, Loss: 0.1229
Batch 59000/76266, Loss: 0.1725
Batch 59100/76266, Loss: 0.2344
Batch 59200/76266, Loss: 0.1981
Batch 59300/76266, Loss: 0.2712
Batch 59400/76266, Loss: 0.2611
Batch 59500/76266, Loss: 0.1922
Batch 59600/76266, Loss: 0.1433
Batch 59700/76266, Loss: 0.1363
Batch 59800/76266, Loss: 0.1079
Batch 59900/76266, Loss: 0.1667
Batch 60000/76266, Loss: 0.1314
Batch 60100/76266, Loss: 0.1151
Batch 60200/76266, Loss: 0.1654
Batch 60300/76266, Loss: 0.2288
Batch 60400/76266, Loss: 0.1977
Batch 60500/76266, Loss: 0.2026
Batch 60600/76266, Loss: 0.1597
Batch 60700/76266, Loss: 0.1973
Batch 60800/76266, Loss: 0.2161
Batch 60900/76266, Loss: 0.1985
Batch 61000/76266, Loss: 0.1962
Batch 61100/76266, Loss: 0.1911
Batch 61200/76266, Loss: 0.1455
Batch 61300/76266, Loss: 0.1559
Batch 61400/76266, Loss: 0.1704
Batch 61500/76266, Loss: 0.1050
Batch 61600/76266, Loss: 0.1968
Batch 61700/76266, Loss: 0.1184
Batch 61800/76266, Loss: 0.1725
Batch 61900/76266, Loss: 0.2429
Batch 62000/76266, Loss: 0.1887
Batch 62100/76266, Loss: 0.1769
Batch 62200/76266, Loss: 0.1583
Batch 62300/76266, Loss: 0.2133
Batch 62400/76266, Loss: 0.2103

Batch 62500/76266, Loss: 0.1944
Batch 62600/76266, Loss: 0.1588
Batch 62700/76266, Loss: 0.2187
Batch 62800/76266, Loss: 0.1659
Batch 62900/76266, Loss: 0.1499
Batch 63000/76266, Loss: 0.2123
Batch 63100/76266, Loss: 0.1439
Batch 63200/76266, Loss: 0.1400
Batch 63300/76266, Loss: 0.1301
Batch 63400/76266, Loss: 0.2148
Batch 63500/76266, Loss: 0.1561
Batch 63600/76266, Loss: 0.1660
Batch 63700/76266, Loss: 0.2050
Batch 63800/76266, Loss: 0.1905
Batch 63900/76266, Loss: 0.1574
Batch 64000/76266, Loss: 0.1262
Batch 64100/76266, Loss: 0.2140
Batch 64200/76266, Loss: 0.1325
Batch 64300/76266, Loss: 0.1259
Batch 64400/76266, Loss: 0.2574
Batch 64500/76266, Loss: 0.1775
Batch 64600/76266, Loss: 0.1369
Batch 64700/76266, Loss: 0.2303
Batch 64800/76266, Loss: 0.2511
Batch 64900/76266, Loss: 0.2576
Batch 65000/76266, Loss: 0.1562
Batch 65100/76266, Loss: 0.2335
Batch 65200/76266, Loss: 0.1713
Batch 65300/76266, Loss: 0.2396
Batch 65400/76266, Loss: 0.1943
Batch 65500/76266, Loss: 0.1388
Batch 65600/76266, Loss: 0.1457
Batch 65700/76266, Loss: 0.2600
Batch 65800/76266, Loss: 0.2374
Batch 65900/76266, Loss: 0.1909
Batch 66000/76266, Loss: 0.1273
Batch 66100/76266, Loss: 0.1083
Batch 66200/76266, Loss: 0.1783
Batch 66300/76266, Loss: 0.1258
Batch 66400/76266, Loss: 0.1221
Batch 66500/76266, Loss: 0.1081
Batch 66600/76266, Loss: 0.2219
Batch 66700/76266, Loss: 0.2143
Batch 66800/76266, Loss: 0.1458
Batch 66900/76266, Loss: 0.2370
Batch 67000/76266, Loss: 0.1763
Batch 67100/76266, Loss: 0.3211
Batch 67200/76266, Loss: 0.1005
Batch 67300/76266, Loss: 0.1318
Batch 67400/76266, Loss: 0.1727
Batch 67500/76266, Loss: 0.0858
Batch 67600/76266, Loss: 0.1719
Batch 67700/76266, Loss: 0.1761
Batch 67800/76266, Loss: 0.0963
Batch 67900/76266, Loss: 0.2359
Batch 68000/76266, Loss: 0.2676
Batch 68100/76266, Loss: 0.1753

Batch 68200/76266, Loss: 0.2483
Batch 68300/76266, Loss: 0.2782
Batch 68400/76266, Loss: 0.1551
Batch 68500/76266, Loss: 0.1431
Batch 68600/76266, Loss: 0.2635
Batch 68700/76266, Loss: 0.2619
Batch 68800/76266, Loss: 0.1467
Batch 68900/76266, Loss: 0.1698
Batch 69000/76266, Loss: 0.1586
Batch 69100/76266, Loss: 0.1671
Batch 69200/76266, Loss: 0.2694
Batch 69300/76266, Loss: 0.1666
Batch 69400/76266, Loss: 0.0849
Batch 69500/76266, Loss: 0.0905
Batch 69600/76266, Loss: 0.1686
Batch 69700/76266, Loss: 0.2019
Batch 69800/76266, Loss: 0.1346
Batch 69900/76266, Loss: 0.1784
Batch 70000/76266, Loss: 0.2171
Batch 70100/76266, Loss: 0.1999
Batch 70200/76266, Loss: 0.1198
Batch 70300/76266, Loss: 0.2445
Batch 70400/76266, Loss: 0.1354
Batch 70500/76266, Loss: 0.1999
Batch 70600/76266, Loss: 0.2178
Batch 70700/76266, Loss: 0.1480
Batch 70800/76266, Loss: 0.2318
Batch 70900/76266, Loss: 0.1666
Batch 71000/76266, Loss: 0.2108
Batch 71100/76266, Loss: 0.2624
Batch 71200/76266, Loss: 0.2166
Batch 71300/76266, Loss: 0.1687
Batch 71400/76266, Loss: 0.2632
Batch 71500/76266, Loss: 0.1660
Batch 71600/76266, Loss: 0.2125
Batch 71700/76266, Loss: 0.1218
Batch 71800/76266, Loss: 0.1678
Batch 71900/76266, Loss: 0.2423
Batch 72000/76266, Loss: 0.1550
Batch 72100/76266, Loss: 0.1755
Batch 72200/76266, Loss: 0.1578
Batch 72300/76266, Loss: 0.0822
Batch 72400/76266, Loss: 0.3154
Batch 72500/76266, Loss: 0.1848
Batch 72600/76266, Loss: 0.1799
Batch 72700/76266, Loss: 0.1095
Batch 72800/76266, Loss: 0.1803
Batch 72900/76266, Loss: 0.1592
Batch 73000/76266, Loss: 0.1501
Batch 73100/76266, Loss: 0.0970
Batch 73200/76266, Loss: 0.1920
Batch 73300/76266, Loss: 0.1435
Batch 73400/76266, Loss: 0.2002
Batch 73500/76266, Loss: 0.1031
Batch 73600/76266, Loss: 0.1995
Batch 73700/76266, Loss: 0.1604

Batch 73800/76266, Loss: 0.1039
Batch 73900/76266, Loss: 0.2055
Batch 74000/76266, Loss: 0.2389
Batch 74100/76266, Loss: 0.1161
Batch 74200/76266, Loss: 0.1586
Batch 74300/76266, Loss: 0.1167
Batch 74400/76266, Loss: 0.2678
Batch 74500/76266, Loss: 0.2579
Batch 74600/76266, Loss: 0.1669
Batch 74700/76266, Loss: 0.1966
Batch 74800/76266, Loss: 0.1052
Batch 74900/76266, Loss: 0.1706
Batch 75000/76266, Loss: 0.1281
Batch 75100/76266, Loss: 0.1134
Batch 75200/76266, Loss: 0.1010
Batch 75300/76266, Loss: 0.1767
Batch 75400/76266, Loss: 0.1817
Batch 75500/76266, Loss: 0.2451
Batch 75600/76266, Loss: 0.1347
Batch 75700/76266, Loss: 0.1413
Batch 75800/76266, Loss: 0.1703
Batch 75900/76266, Loss: 0.2593
Batch 76000/76266, Loss: 0.1349
Batch 76100/76266, Loss: 0.1150
Batch 76200/76266, Loss: 0.1547

Epoch 3 completed in 11576.09s. Average Training Loss: 0.1810

Transformer training finished.

```
# --- Save the Model and Tokenizer ---
print("\nSaving the fine-tuned model and tokenizer...")

# Define the directory where you want to save them
drive.mount('/content/drive') # Mount Google Drive
save_directory = '/content/drive/My Drive/Colab Notebooks' # You can change this path
# Save the model's weights and configuration file
model.save_pretrained(save_directory)

# Save the tokenizer's vocabulary and configuration file
tokenizer.save_pretrained(save_directory)

print(f"Model and tokenizer saved to {save_directory}")

# --- (Optional) How to load the model and tokenizer later ---
# from transformers import AutoModelForSequenceClassification, AutoTokenizer
# loaded_model = AutoModelForSequenceClassification.from_pretrained(save_directory)
# loaded_tokenizer = AutoTokenizer.from_pretrained(save_directory)
# print("\nModel and tokenizer loaded successfully (example).")
# loaded_model.to(device) # Remember to move the loaded model to the correct device
```



Saving the fine-tuned model and tokenizer...

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount()
Model and tokenizer saved to /content/drive/My Drive/Colab Notebooks



```
print("\nEvaluating Transformer Model...")
model.eval() # Set model to evaluation mode

all_preds_transformer = []
all_labels_transformer = []

with torch.no_grad(): # Disable gradient calculations for inference
    for batch in test_loader:
        # Move batch to device
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device) # Shape: (batch_size, num_aspects)

        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs.logits # Shape: (batch_size, num_aspects * num_classes_per_aspect)

        # Get predictions: Reshape logits and find the class with max probability for each aspect
        # Reshape logits to (batch_size, num_aspects, num_classes_per_aspect)
        reshaped_logits = logits.view(input_ids.size(0), num_aspects, num_classes_per_aspect)
        predictions = torch.argmax(reshaped_logits, dim=2) # Get predicted class index along last dimension

    all_preds_transformer.extend(predictions.cpu().numpy())
    all_labels_transformer.extend(labels.cpu().numpy())
```

```

# Convert collected predictions and labels into numpy arrays
Y_pred_transformer_np = np.array(all_preds_transformer) # Shape: (num_test_samples, num_asp
Y_true_transformer_np = np.array(all_labels_transformer) # Shape: (num_test_samples, num_as

# Calculate metrics per aspect
transformer_report = {}
print("\n--- Transformer Classification Report (Per Aspect) ---")

all_true_flat_transformer = []
all_pred_flat_transformer = []

# Assuming 'aspects' is defined earlier and includes 'overall'
# aspects = ['food', 'service', 'ambiance', 'price', 'context', 'overall']

for i, aspect_name in enumerate(aspects): # Iterate through all aspects, including 'overall'
    print(f"\n--- Aspect: {aspect_name} ---")
    true_labels = Y_true_transformer_np[:, i]
    pred_labels = Y_pred_transformer_np[:, i]

    all_true_flat_transformer.extend(true_labels)
    all_pred_flat_transformer.extend(pred_labels)

# Get unique labels in pred_labels and true_labels
unique_labels = np.unique(np.concatenate((pred_labels, true_labels)))

# Filter target_names to include only the present labels
present_target_names = [name for idx, name in enumerate(sentiment_classes) if idx in uni

report = classification_report(true_labels, pred_labels, target_names=present_target_name
print(report)
precision, recall, f1, _ = precision_recall_fscore_support(true_labels, pred_labels, ave
accuracy = accuracy_score(true_labels, pred_labels)
transformer_report[aspect_name] = {'precision': precision, 'recall': recall, 'f1-score':

# Overall Micro and Macro Averages
print("\n--- Transformer Overall Micro/Macro Averages ---")
precision_micro_t, recall_micro_t, f1_micro_t, _ = precision_recall_fscore_support(all_true_
precision_macro_t, recall_macro_t, f1_macro_t, _ = precision_recall_fscore_support(all_true_
print(f"Micro Average: Precision={precision_micro_t:.4f}, Recall={recall_micro_t:.4f}, F1-Sc
print(f"Macro Average: Precision={precision_macro_t:.4f}, Recall={recall_macro_t:.4f}, F1-Sc

transformer_report['overall_micro'] = {'precision': precision_micro_t, 'recall': recall_micr
transformer_report['overall_macro'] = {'precision': precision_macro_t, 'recall': recall_macr

# Store transformer results for comparison plots
transformer_f1_scores = {aspect: metrics['f1-score'] for aspect, metrics in transformer_repc

```



Evaluating Transformer Model...

--- Transformer Classification Report (Per Aspect) ---



```

--- Aspect: food ---
      precision    recall  f1-score   support

negative    0.66      0.63      0.65     24235
neutral     0.85      0.74      0.79     77023
positive    0.90      0.95      0.92    203805

accuracy                    0.87     305063
macro avg    0.80      0.77      0.79     305063
weighted avg 0.87      0.87      0.87     305063

```

```

--- Aspect: service ---
      precision    recall  f1-score   support

negative    0.73      0.74      0.74     25208
neutral     0.91      0.89      0.90    187376
positive    0.81      0.85      0.83     92479

accuracy                    0.86     305063
macro avg    0.82      0.83      0.82     305063
weighted avg 0.87      0.86      0.86     305063

```

```

--- Aspect: ambiance ---
      precision    recall  f1-score   support

negative    0.62      0.49      0.55      3174
neutral     0.97      0.97      0.97    270947
positive    0.76      0.77      0.77     30942

accuracy                    0.95     305063
macro avg    0.78      0.75      0.76     305063
weighted avg 0.94      0.95      0.95     305063

```

```

--- Aspect: price ---
      precision    recall  f1-score   support

negative    0.56      0.34      0.43      3099
neutral     0.97      0.98      0.98    286154
positive    0.67      0.62      0.65     15810

accuracy                    0.96     305063
macro avg    0.74      0.65      0.68     305063
weighted avg 0.95      0.96      0.96     305063

```

```

--- Aspect: context ---
      precision    recall  f1-score   support

negative    0.57      0.44      0.50      5564
neutral     0.95      0.95      0.95    253527

```

```

# --- Define the sentiment classes corresponding to your integer labels ---
# This should match the 'bin_labels' used during binning (e.g., 0, 1, 2)
# and the 'label_names' (e.g., 'negative', 'neutral', 'positive')
sentiment_classes = ['negative', 'neutral', 'positive'] # MUST match the order used for bin_
integer_to_string_label_map = { i: label for i, label in enumerate(sentiment_classes) }
print(f"Integer-to-String Label Map for Prediction: {integer_to_string_label_map}")

def predict_sentiment(review_text, model, tokenizer, aspects, int_to_str_map):
    """Predicts sentiment for all aspects for a given review text."""

    # Preprocess the input text
    processed_text = preprocess_text(review_text) # Assumes preprocess_text is defined elsew
    if not processed_text:
        return {"error": "Review text is empty after preprocessing."}

    # Tokenize
    inputs = tokenizer(processed_text, return_tensors='pt', truncation=True, padding=True, m

    # Move inputs to the same device as the model
    inputs = {k: v.to(model.device) for k, v in inputs.items()}

    # Predict
    model.eval() # Ensure model is in eval mode
    with torch.no_grad():
        outputs = model(**inputs)
        logits = outputs.logits # Shape: (1, num_aspects * num_classes_per_aspect)

    # Process logits to get predictions per aspect
    num_classes_per_aspect = len(int_to_str_map) # Get number of classes (e.g., 3)
    # Reshape to (1, num_aspects, num_classes_per_aspect)
    reshaped_logits = logits.view(1, len(aspects), num_classes_per_aspect)

    # Get the index of the max logit for each aspect (these are your 0, 1, 2 labels)
    predictions_indices = torch.argmax(reshaped_logits, dim=2).squeeze().cpu().numpy() # Sha

    # Handle case where there's only one aspect (numpy might return a scalar)
    if predictions_indices.ndim == 0:
        predictions_indices = [predictions_indices.item()] # Make it a list

    # Map predicted integer indices back to sentiment string labels
    predicted_sentiments = {}
    for i, pred_idx in enumerate(predictions_indices):
        # Ensure the predicted index exists in the map, handle potential errors
        sentiment_label = int_to_str_map.get(pred_idx, "Unknown Label")
        predicted_sentiments[aspects[i]] = sentiment_label

    # # --- OLD MAPPING LOGIC (REMOVE/REPLACE) ---
    # # Map indices back to sentiment labels
    # # Create inverse map (0: 'neutral', 1: 'positive', -1: 'negative') <--- This was likel
    # # inverse_label_map = {v: k for k, v in label_map.items()} <--- Remove this line

```

```

# # predicted_sentiments = {aspects[i]: inverse_label_map[pred_idx] for i, pred_idx in enumerate(predicted_results)}
# # --- END OLD MAPPING LOGIC ---

return predicted_sentiments

# --- Example Usage (Location: Near the end of the notebook, around page 20) ---

# Ensure 'aspects' list is defined correctly (should match training)
aspects = ['food', 'service', 'ambiance', 'price', 'context', 'overall']

# Define the mapping from integer labels (0, 1, 2) to strings ('negative', 'neutral', 'positive')
# This map MUST align with the 'bin_labels' used during binning.
# If bin_labels = [0, 1, 2] corresponds to ['negative', 'neutral', 'positive']
integer_to_string_label_map = {
    0: 'negative',
    1: 'neutral',
    2: 'positive'
}

print(f"\nUsing label map for prediction: {integer_to_string_label_map}")

new_review = "The pizza was amazing, truly authentic Italian style! However, the waiter was rude."
# Pass the correct integer-to-string map to the function
predicted_results = predict_sentiment(new_review, model, tokenizer, aspects, integer_to_string_label_map)

print("\n--- Example Prediction ---")
print(f"Review: \"{new_review}\"")
print("\nPredicted Sentiments:")
if "error" in predicted_results:
    print(predicted_results["error"])
else:
    for aspect, sentiment in predicted_results.items():
        print(f"- {aspect.capitalize()}: {sentiment.capitalize()}") # Keep .capitalize() for consistency

new_review_2 = "Service was terrible, and the food was bad and pricey. Terrible atmosphere."
predicted_results_2 = predict_sentiment(new_review_2, model, tokenizer, aspects, integer_to_string_label_map)
print("\n--- Example Prediction 2 ---")
print(f"Review: \"{new_review_2}\"")
print("\nPredicted Sentiments:")
if "error" in predicted_results_2:
    print(predicted_results_2["error"])
else:
    for aspect, sentiment in predicted_results_2.items():
        print(f"- {aspect.capitalize()}: {sentiment.capitalize()}")

new_review_3 = "This restaurant is bad."
predicted_results_3 = predict_sentiment(new_review_3, model, tokenizer, aspects, integer_to_string_label_map)
print("\n--- Example Prediction 3 ---") # Corrected print statement index
print(f"Review: \"{new_review_3}\"")
print("\nPredicted Sentiments:")

```

```

if "error" in predicted_results_3:
    print(predicted_results_3["error"])
else:
    for aspect, sentiment in predicted_results_3.items():
        print(f"- {aspect.capitalize()}: {sentiment.capitalize()}")

new_review_4 = "This restaurant is great. The food was yummy. The service was excellent. The
predicted_results_4 = predict_sentiment(new_review_4, model, tokenizer, aspects, integer_to_
print("\n--- Example Prediction 4 ---") # Corrected print statement index
print(f"Review: \"{new_review_4}\"")
print("\nPredicted Sentiments:")
if "error" in predicted_results_4:
    print(predicted_results_4["error"])
else:
    for aspect, sentiment in predicted_results_4.items():
        print(f"- {aspect.capitalize()}: {sentiment.capitalize()}")

```

➡ Integer-to-String Label Map for Prediction: {0: 'negative', 1: 'neutral', 2: 'positive'}

Using label map for prediction: {0: 'negative', 1: 'neutral', 2: 'positive'}

--- Example Prediction ---

Review: "The pizza was amazing, truly authentic Italian style! However, the waiter was c

Predicted Sentiments:

- Food: Positive
- Service: Negative
- Ambiance: Neutral
- Price: Neutral
- Context: Neutral
- Overall: Positive

--- Example Prediction 2 ---

Review: "Service was terrible, and the food was bad and pricey. Terrible atmosphere."

Predicted Sentiments:

- Food: Neutral
- Service: Negative
- Ambiance: Negative
- Price: Neutral
- Context: Neutral
- Overall: Negative

--- Example Prediction 3 ---

Review: "This restaurant is bad."

Predicted Sentiments:

- Food: Neutral
- Service: Negative
- Ambiance: Neutral
- Price: Neutral
- Context: Neutral

- Overall: Negative

--- Example Prediction 4 ---

Review: "This restaurant is great. The food was yummy. The service was excellent. The pr

Predicted Sentiments:

- Food: Positive
- Service: Neutral
- Ambiance: Positive
- Price: Neutral
- Context: Neutral
- Overall: Positive

--- Get User Input and Predict ---

while True: